Berkeley MSSE
MOLECULAR SCIENCE & SOFTWARE ENGINEERING

Lecture 15:

Introduction to Artificial Neural Networks

Markus Hohle

University of California, Berkeley

Data Science for Scientific Computing

MSSE 277A, 3 Units

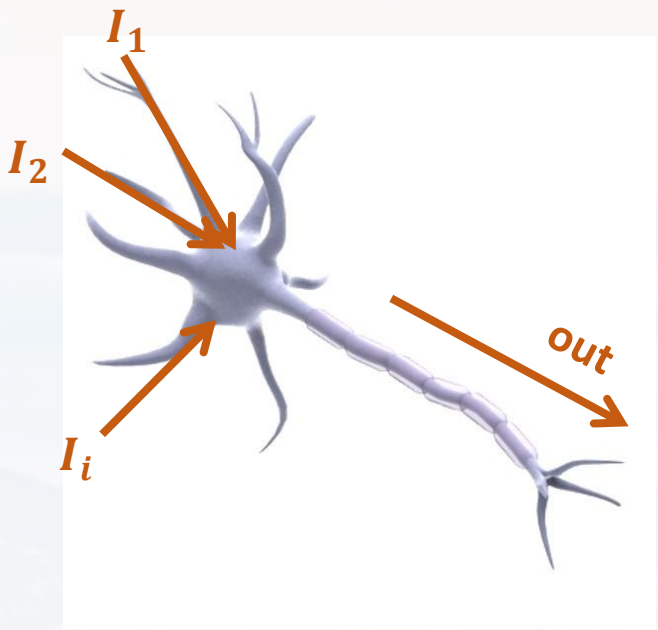
Outline

- **The Perceptron**
- **Gradient Descent & Backpropagation**
- **Multiple Dense Layers**
- **Examples in Keras and PyTorch**
- **Feature Selection & Regularization** (again)
- **Other Architectures**

Outline

- **The Perceptron**
- Gradient Descent & Backpropagation
- Multiple Dense Layers
- Examples in Keras and PyTorch
- Feature Selection & Regularization (again)
- Other Architectures

Introduction to Artificial Neural Networks: The Perceptron



what we know…

- inputs enter the neuron
- something happens inside the neuron
- neuron generates an output

…how we could model it

- inputs must be weighted (important vs unimportant input)
- **learning process: changing weights**
- output = sum of weighted inputs

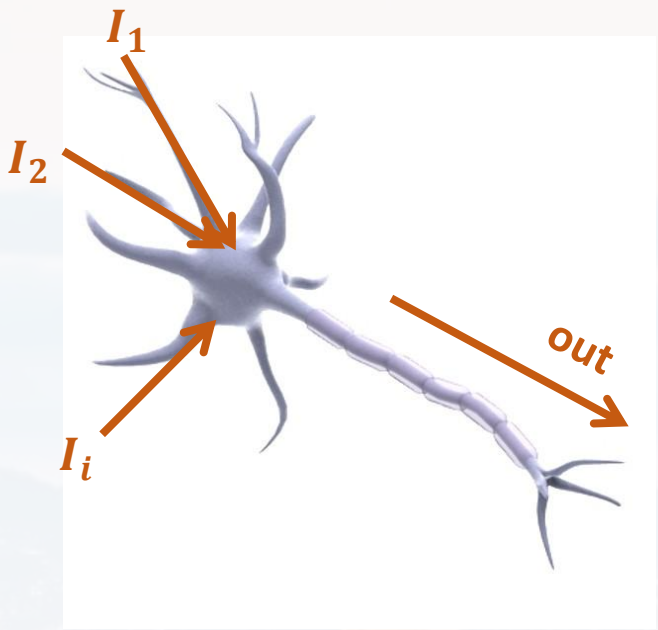
I_i input *i*

w_i corresponding weight

b bias (base potential)

$$net = \underbrace{\sum_i I_i \cdot w_i}_{\textit{dot product}} + b$$

Introduction to Artificial Neural Networks: The Perceptron



I_i	input *i*
w_i	corresponding weight
b	bias (base potential)

$$net = \underbrace{\sum_i I_i \cdot w_i}_{\textbf{dot product}} + b$$

recall: linear models

$$y_k = \beta_0 + \sum_{n=1}^{N} \beta_n x_{kn} + \epsilon$$

$$\begin{pmatrix} y_1 \\ \dots \\ y_k \\ \dots \\ \dots \\ y_K \end{pmatrix} = \begin{pmatrix} 1 & x_{11} & x_{12} & \dots & x_{1n} & \dots & x_{1N} \\ \dots & \dots & & & \dots & & \\ 1 & x_{k1} & & & x_{kn} & & \\ 1 & \dots & & & \dots & & \\ 1 & x_{K1} & x_{K2} & \dots & x_{Kn} & \dots & x_{KN} \end{pmatrix} \begin{pmatrix} \beta_0 \\ \beta_1 \\ \dots \\ \beta_n \\ \dots \\ \beta_N \end{pmatrix} + \begin{pmatrix} \varepsilon_1 \\ \varepsilon_2 \\ \dots \\ \varepsilon_k \\ \dots \\ \varepsilon_K \end{pmatrix}$$

I_i are the features (correspond to x_{kn})
w_i correspond to β_n
b corresponds to β_0

Introduction to Artificial Neural Networks: The Perceptron

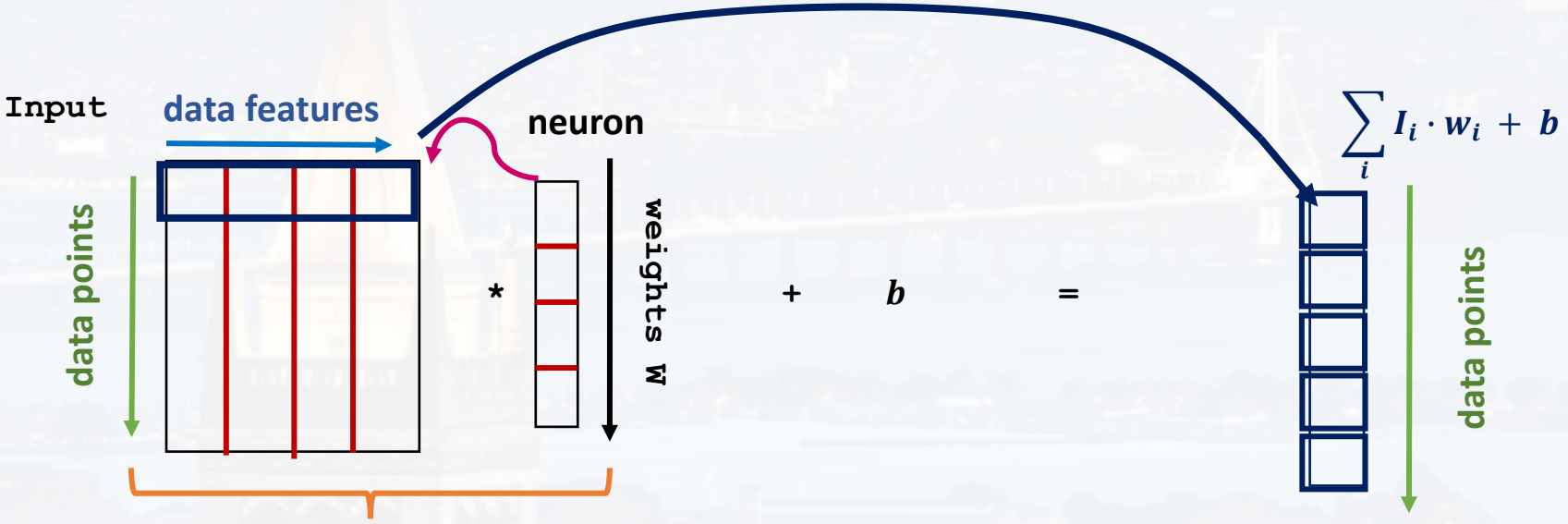
data features

data points

```
[[5.1 3.5 1.4 0.2]
 [4.9 3.  1.4 0.2]
 [4.7 3.2 1.3 0.2]
 [4.6 3.1 1.5 0.2]
 [5.  3.6 1.4 0.2]
 [5.4 3.9 1.7 0.4]
 [4.6 3.4 1.4 0.3]
 [5.  3.4 1.5 0.2]
```

$$net = \sum_i I_i \cdot w_i + b$$

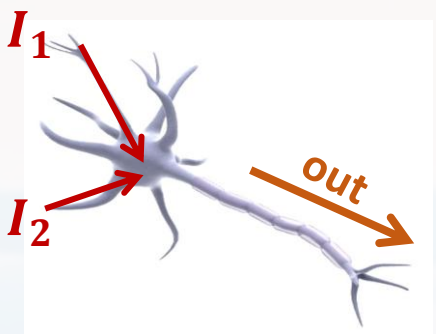
dot product



`np.dot(Input, W)`

`np.dot(Input, W)`

Introduction to Artificial Neural Networks: The Perceptron



still just a linear model!

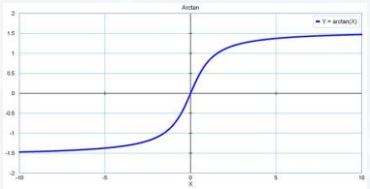
$$net = \sum_i I_i \cdot w_i + b$$

I_i **input *i***
w_i **corresponding weight**
b **bias (base potential)**

$$\sum_i I_i \cdot w_i + b = net$$

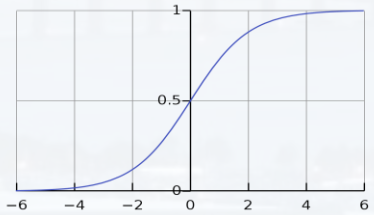
→ some activation function f → actual output

- $y = actan(net)$



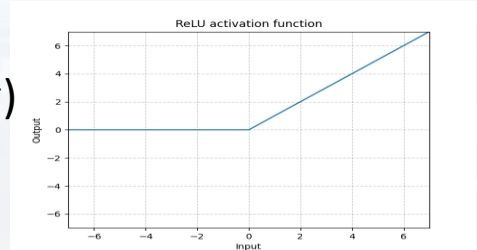
$(-\infty;\ +\infty) \rightarrow (-\pi/2;\ +\pi/2)$

- y = sigm(net)



$(-\infty;\ +\infty) \rightarrow (0; 1)$

- y = ReLU(net)
= *max(0, net)*

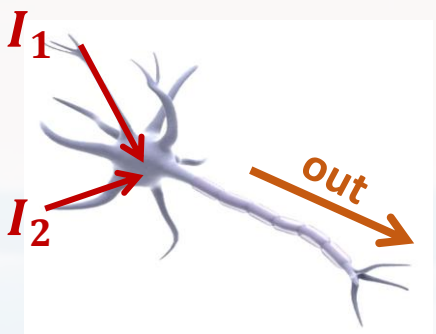


$(-\infty;\ +\infty) \rightarrow (0; +\infty)$

Outline

- The Perceptron
- **Gradient Descent & Backpropagation**
- Multiple Dense Layers
- Examples in Keras and PyTorch
- Feature Selection & Regularization (again)
- Other Architectures

Introduction to Artificial Neural Networks: Gradient Descent & Backpropagation



still just a linear model!

$$\boxed{net = \sum_i I_i \cdot w_i + b}$$

I_i input i
w_i corresponding weight
b bias (base potential)

$$y = f(net)$$

the forward part:

$$net = \sum_i I_i \cdot w_i + b \xrightarrow{\text{activation function } f} y = f(net) \xrightarrow{\text{objective function } E} E = \frac{1}{2}(t - y)^2$$

target output t

finding best w_i by **minimizing** the **loss function** E

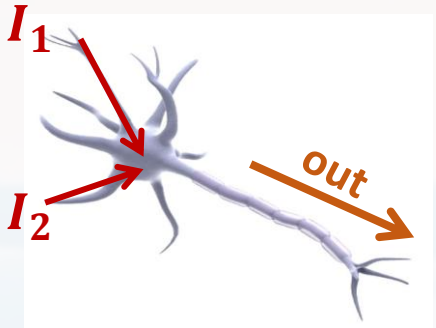
$$\Delta w_i = w_i(new) - w_i(old) = -\alpha \frac{\partial E}{\partial w_i}$$

gradient descent (see last module) with learning rate α

$$\frac{\partial E}{\partial w_i} = \frac{\partial E}{\partial y}\frac{\partial y}{\partial net}\frac{\partial net}{\partial w_i}$$

chain rule

Introduction to Artificial Neural Networks: Gradient Descent & Backpropagation



still just a linear model!

$$\boxed{net = \sum_i I_i \cdot w_i + b}$$

I_i input i
w_i corresponding weight
b bias (base potential)

$$y = f(net)$$

the forward part:

$$net = \sum_i I_i \cdot w_i + b \xrightarrow{\text{activation function } f} y = f(net) \xrightarrow{\text{objective function } E} E = \frac{1}{2}(t - y)^2$$

target output t

finding best w_i by **minimizing** the **loss function** E

$$\Delta w_i = w_i(new) - w_i(old) = -\alpha \frac{\partial E}{\partial w_i}$$

gradient descent (see last module) with learning rate α

$$\frac{\partial E}{\partial w_i} = \frac{\partial E}{\partial y}\frac{\partial y}{\partial net}\frac{\partial net}{\partial w_i} = -(t - y)\, f'(net)\, I_i$$

Introduction to Artificial Neural Networks: Gradient Descent & Backpropagation

the forward part:

activation function ***f*** objective function ***E*** target output ***t***

$$net = \sum_i I_i \cdot w_i + b \longrightarrow y = f(net) \longrightarrow E = \frac{1}{2}(t - y)^2$$

finding best w_i by **minimizing** the **loss function** E

$$\Delta \boldsymbol{w_i} = \boldsymbol{w_i}(\boldsymbol{new}) - \boldsymbol{w_i}(\boldsymbol{old}) = -\boldsymbol{\alpha}\frac{\boldsymbol{\partial E}}{\boldsymbol{\partial w_i}}$$

gradient descent with learning rate $\boldsymbol{\alpha}$

$$\frac{\boldsymbol{\partial E}}{\boldsymbol{\partial w_i}} = \frac{\boldsymbol{\partial E}}{\boldsymbol{\partial y}}\frac{\boldsymbol{\partial y}}{\boldsymbol{\partial net}}\frac{\boldsymbol{\partial net}}{\boldsymbol{\partial w_i}} = \boxed{-(\boldsymbol{t} - \boldsymbol{y})\ \boldsymbol{f}'(\boldsymbol{net})}\boxed{\boldsymbol{I_i}}$$

inner derivative

outer derivatives

$\boldsymbol{f}'(\boldsymbol{net})$ depends on the activation function

The required change of ***E*** propagates back to the changes of $\boldsymbol{w_i}$ → **backpropagation**

Introduction to Artificial Neural Networks: Gradient Descent & Backpropagation

the forward part:

activation function ***f*** — objective function ***E*** — target output ***t***

$$net = \sum_i I_i \cdot w_i + b \longrightarrow y = f(net) \longrightarrow E = \frac{1}{2}(t - y)^2$$

finding best w_i by **minimizing** the **loss function** E

$$\Delta \boldsymbol{w_i} = \boldsymbol{w_i(new)} - \boldsymbol{w_i(old)} = -\boldsymbol{\alpha}\frac{\boldsymbol{\partial E}}{\boldsymbol{\partial w_i}}$$

gradient descent with learning rate $\boldsymbol{\alpha}$

$$\frac{\boldsymbol{\partial E}}{\boldsymbol{\partial w_i}} = \frac{\boldsymbol{\partial E}}{\boldsymbol{\partial y}}\frac{\boldsymbol{\partial y}}{\boldsymbol{\partial net}}\frac{\boldsymbol{\partial net}}{\boldsymbol{\partial w_i}} = \boxed{-(\boldsymbol{t} - \boldsymbol{y})\ \boldsymbol{f'(net)}}\boxed{\boldsymbol{I_i}}$$

inner derivative — outer derivatives

The required change of ***E*** propagates back to the changes of $\boldsymbol{w_i}$ → **backpropagation**

$$\boxed{\Delta \boldsymbol{w_i} = \boldsymbol{w_i(new)} - \boldsymbol{w_i(old)} = \boldsymbol{\alpha}\,(\boldsymbol{t} - \boldsymbol{y})\ \boldsymbol{f'(net)}\ \boldsymbol{I_i}}$$

$$\boxed{\Delta \boldsymbol{b} = \boldsymbol{b(new)} - \boldsymbol{b(old)} \quad = \boldsymbol{\alpha}\,(\boldsymbol{t} - \boldsymbol{y})\ \boldsymbol{f'(net)}\ \cdot \mathbf{1}}$$

Introduction to Artificial Neural Networks: Gradient Descent & Backpropagation

the forward part:

$$\sum_i I_i \cdot w_i + b = net \xrightarrow{\text{activation function } \boldsymbol{f}} y = f(net) \xrightarrow{\text{objective function } \boldsymbol{E}} E = \frac{1}{2}(t - y)^2$$

target output $\boldsymbol{t}$

the backward part:

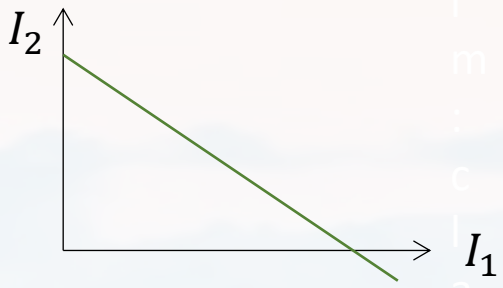
$$\frac{\partial E}{\partial w_i} = \frac{\partial E}{\partial y}\frac{\partial y}{\partial net}\frac{\partial net}{\partial w_i} = -(t - y)\ f'(net)\ I_i$$

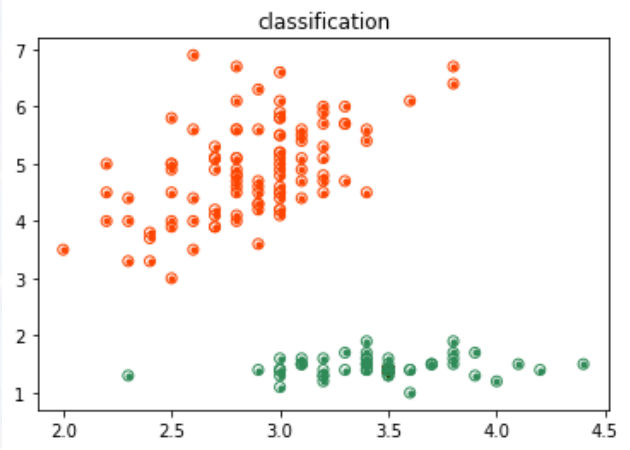
$$\Delta w_i = w_i(new) - w_i(old) = \alpha\,(t - y)\ f'(net)\ I_i$$

$$\Delta b = b(new) - b(old) \quad = \alpha\,(t - y)\ f'(net)\ \cdot 1$$

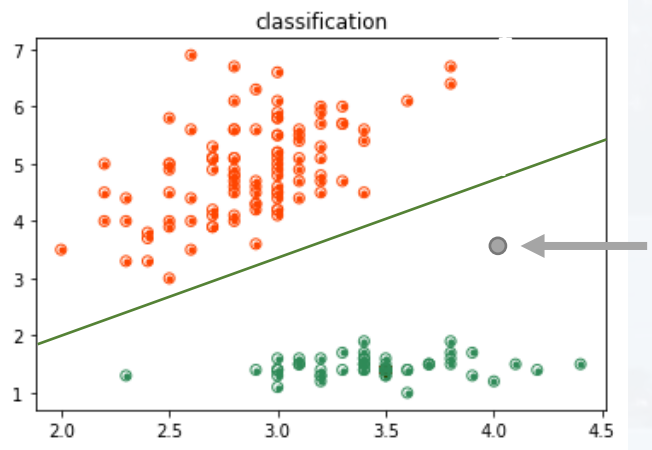
Introduction to Artificial Neural Networks: Gradient Descent & Backpropagation

one neuron
two features
two classes
N data points





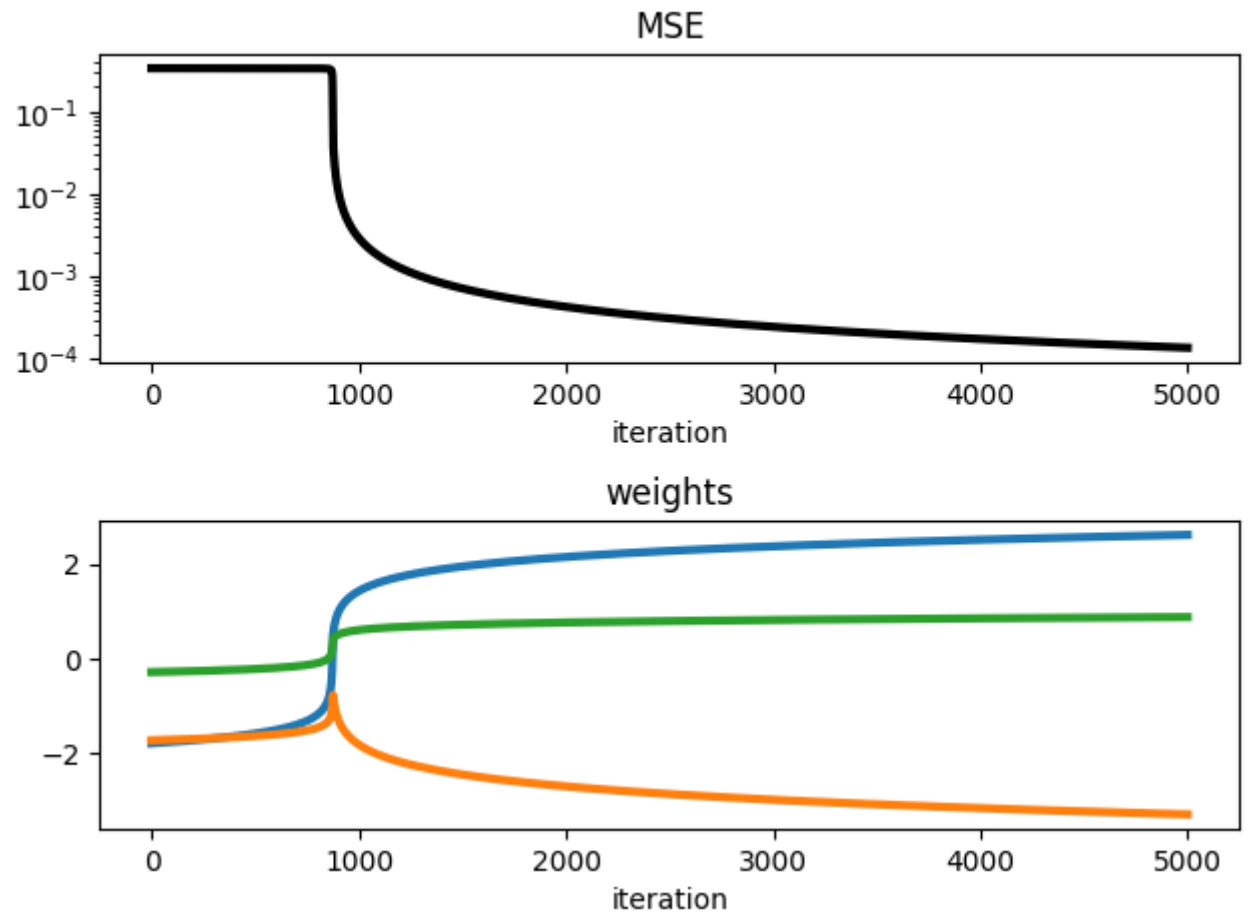
goal

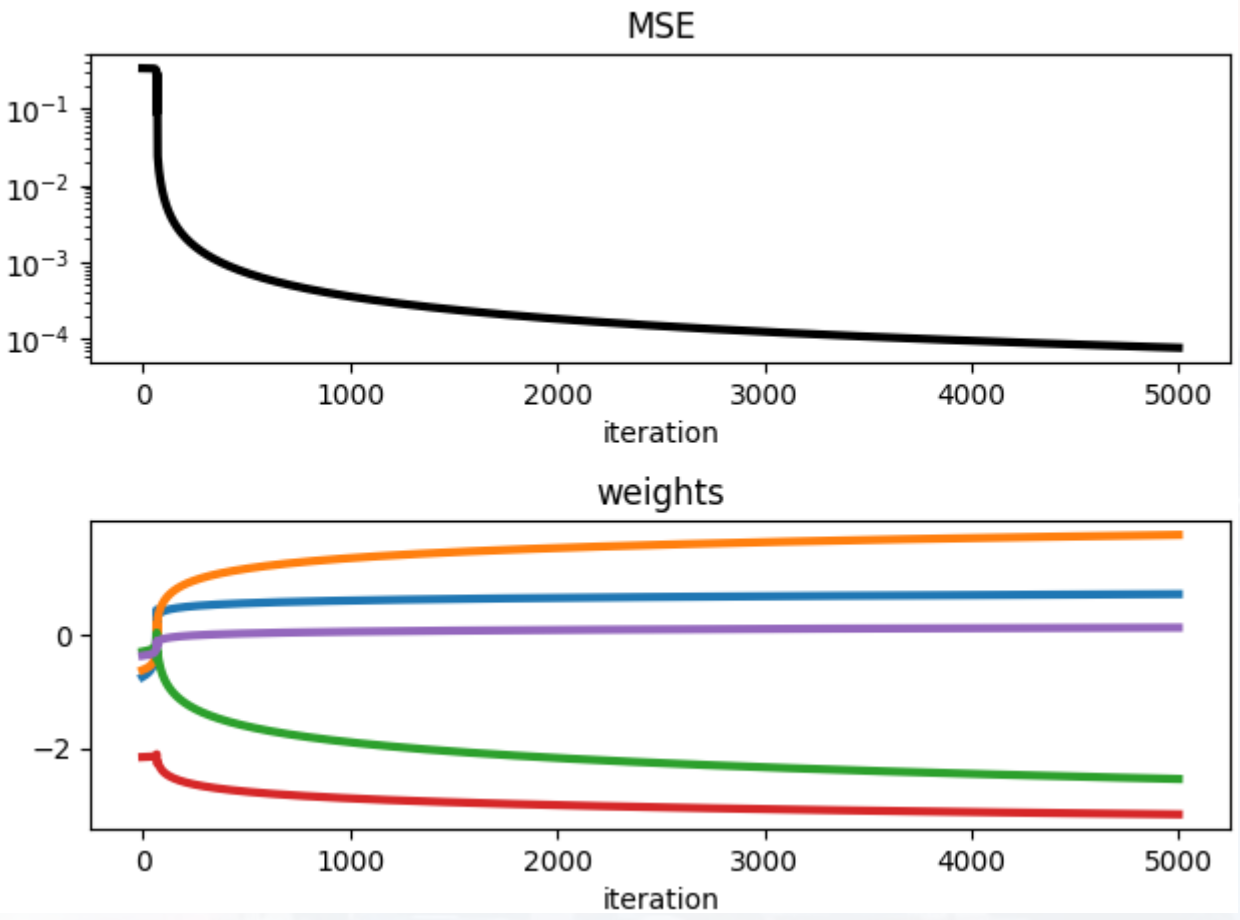


classifying a new data point

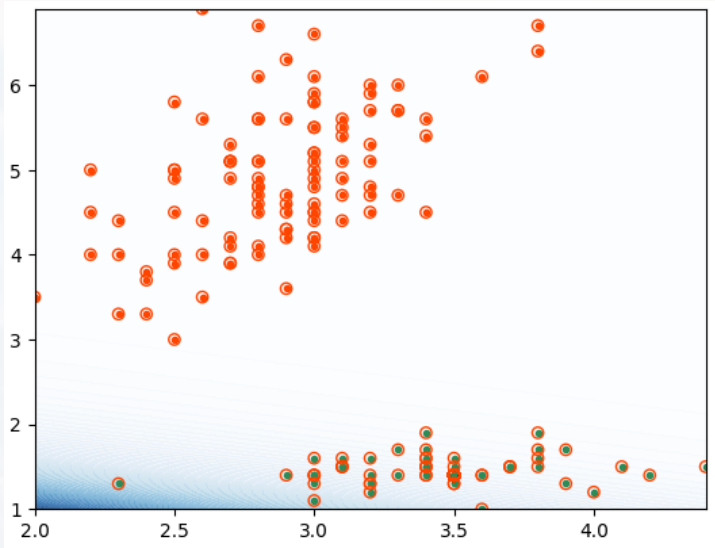
see the Jupyter Notebook `Perceptron.ipynb` for details

The training process

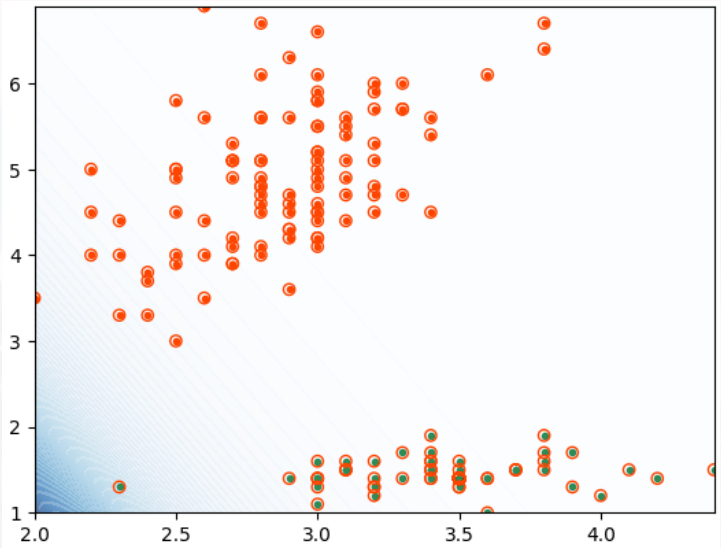




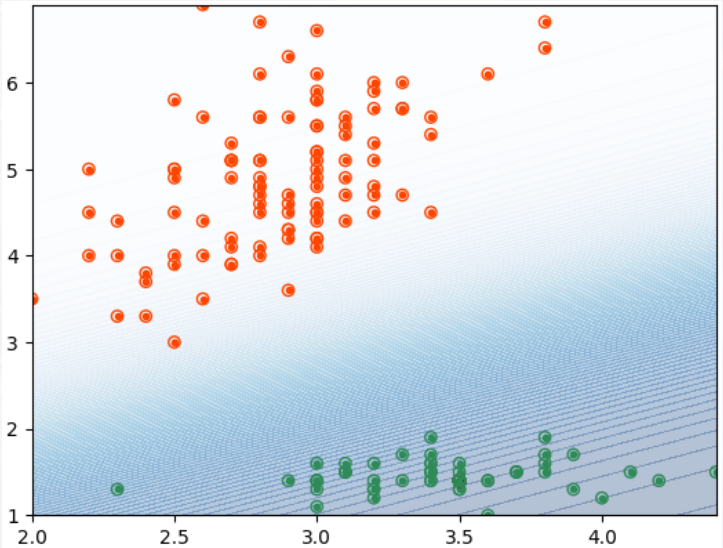
The training process



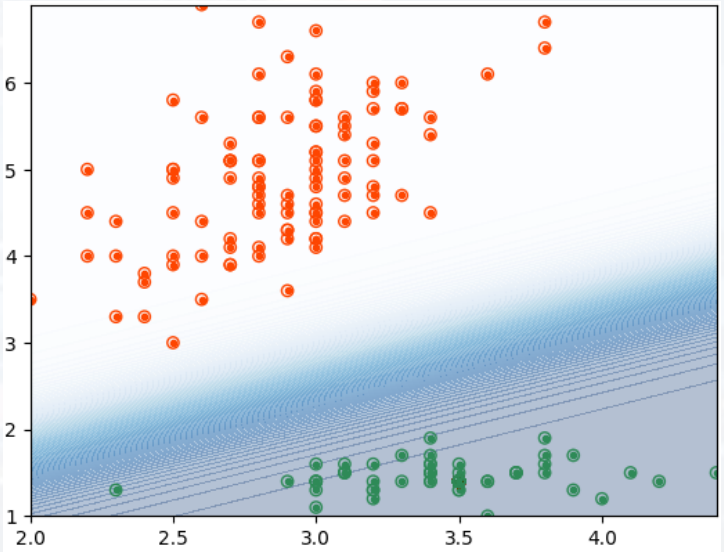
t = 3



t = 5



t = 100



t = 10,000

Outline

- The Perceptron
- Gradient Descent & Backpropagation
- **Multiple Dense Layers**
- Examples in Keras and PyTorch
- Feature Selection & Regularization (again)
- Other Architectures

Introduction to Artificial Neural Networks: Multiple Dense Layers

data features

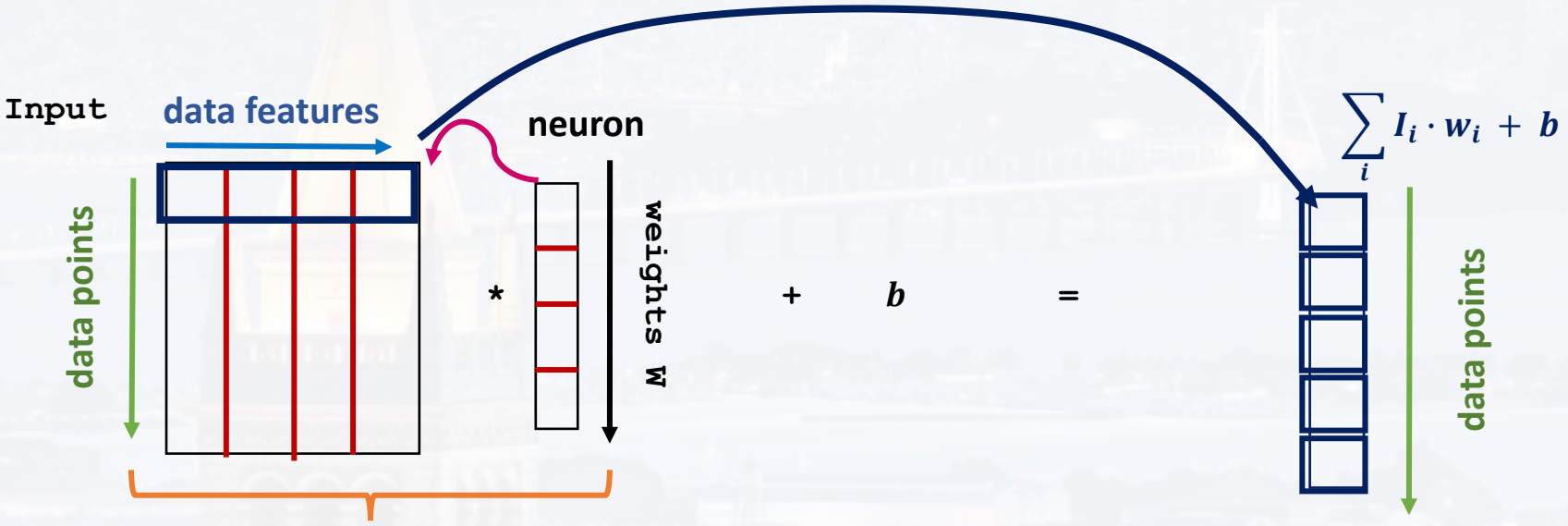
data points

```
[[5.1 3.5 1.4 0.2]
 [4.9 3.  1.4 0.2]
 [4.7 3.2 1.3 0.2]
 [4.6 3.1 1.5 0.2]
 [5.  3.6 1.4 0.2]
 [5.4 3.9 1.7 0.4]
 [4.6 3.4 1.4 0.3]
 [5.  3.4 1.5 0.2]
```

$$net = \sum_i I_i \cdot w_i + b$$

dot product

one single neuron



`np.dot(Input, W)`

`np.dot(Input, W)`

Introduction to Artificial Neural Networks: Multiple Dense Layers

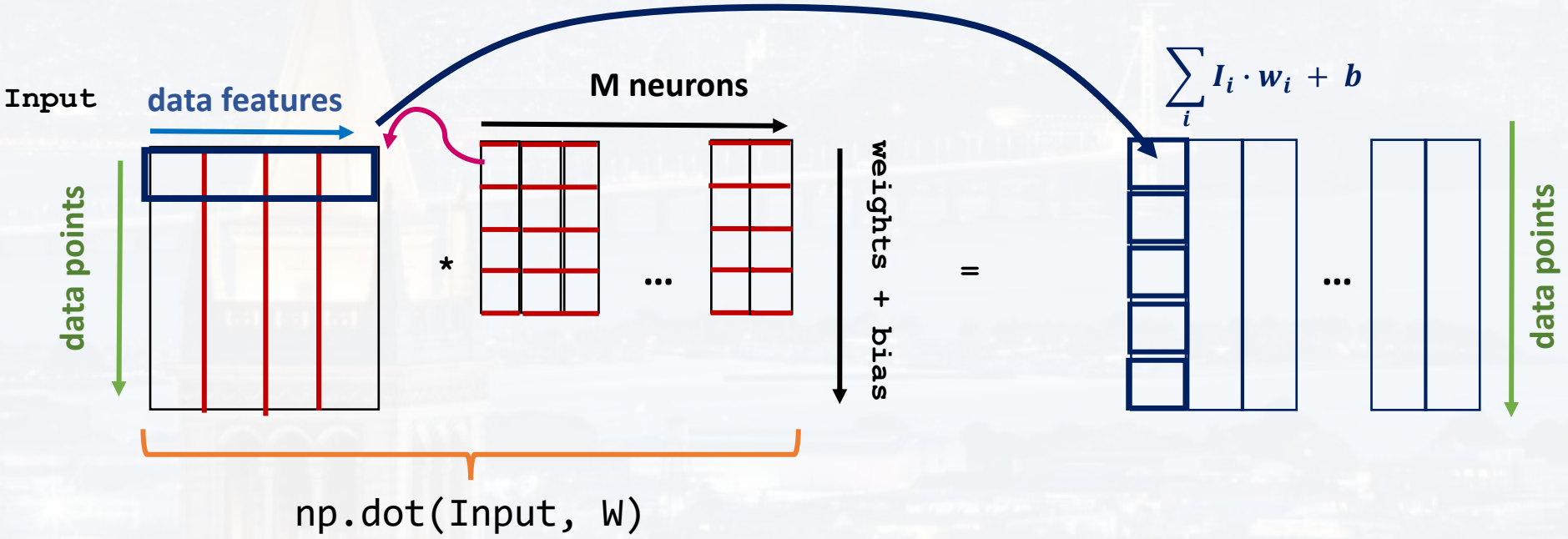
data features

data points

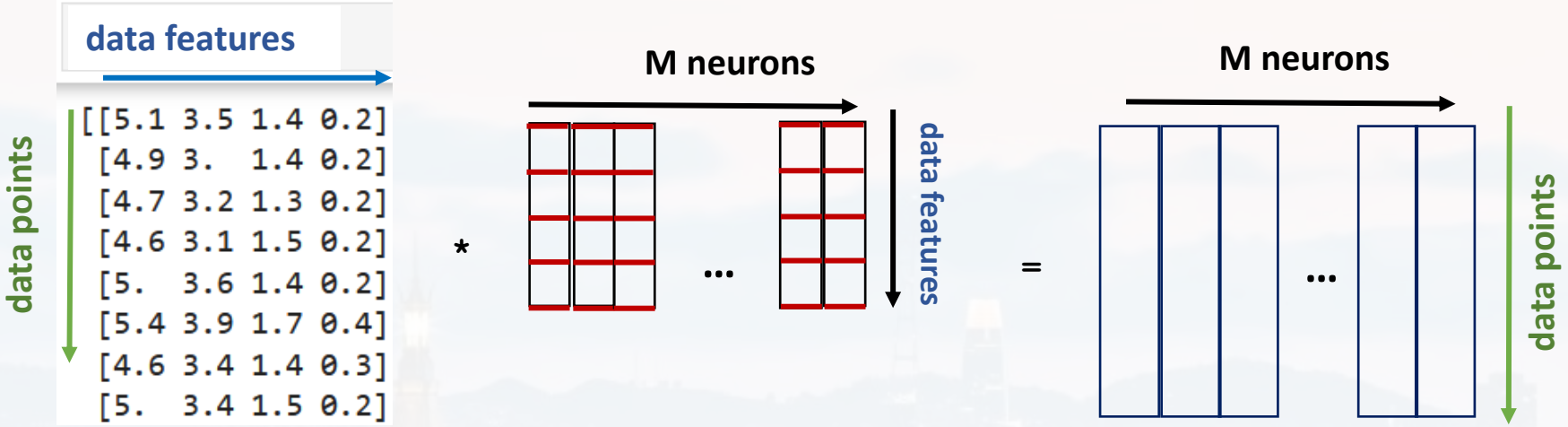
```
[[5.1 3.5 1.4 0.2]
 [4.9 3.  1.4 0.2]
 [4.7 3.2 1.3 0.2]
 [4.6 3.1 1.5 0.2]
 [5.  3.6 1.4 0.2]
 [5.4 3.9 1.7 0.4]
 [4.6 3.4 1.4 0.3]
 [5.  3.4 1.5 0.2]
```

$$net = \sum_i I_i \cdot w_i + b$$

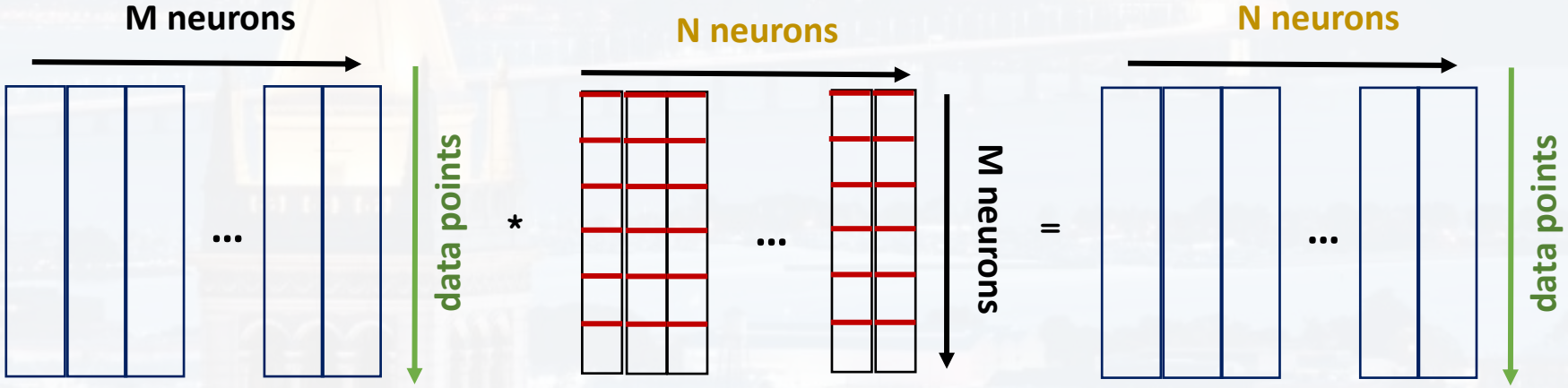
dot product



Introduction to Artificial Neural Networks: Multiple Dense Layers

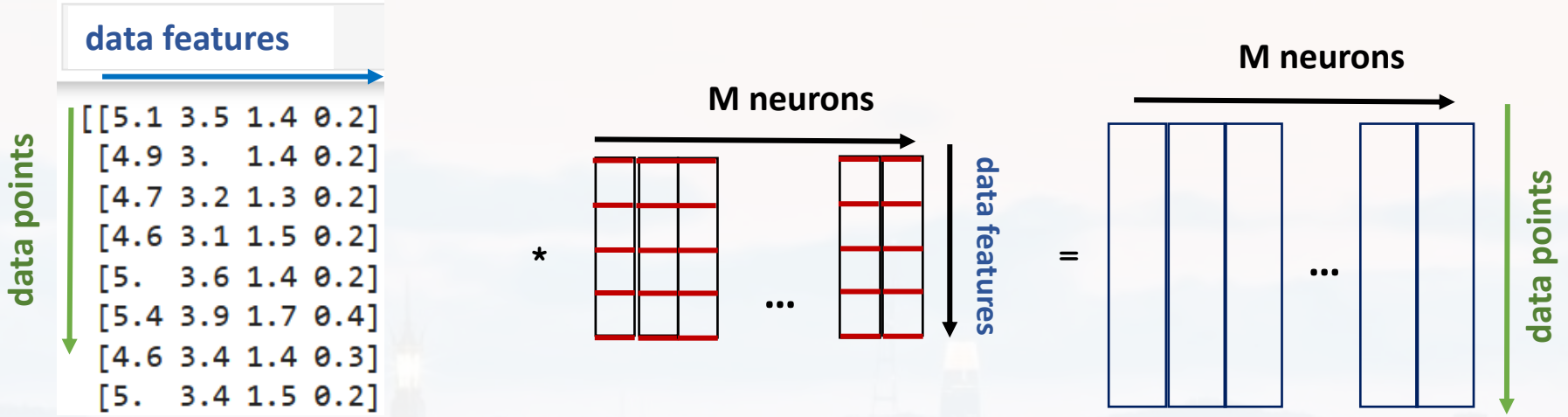


input layer

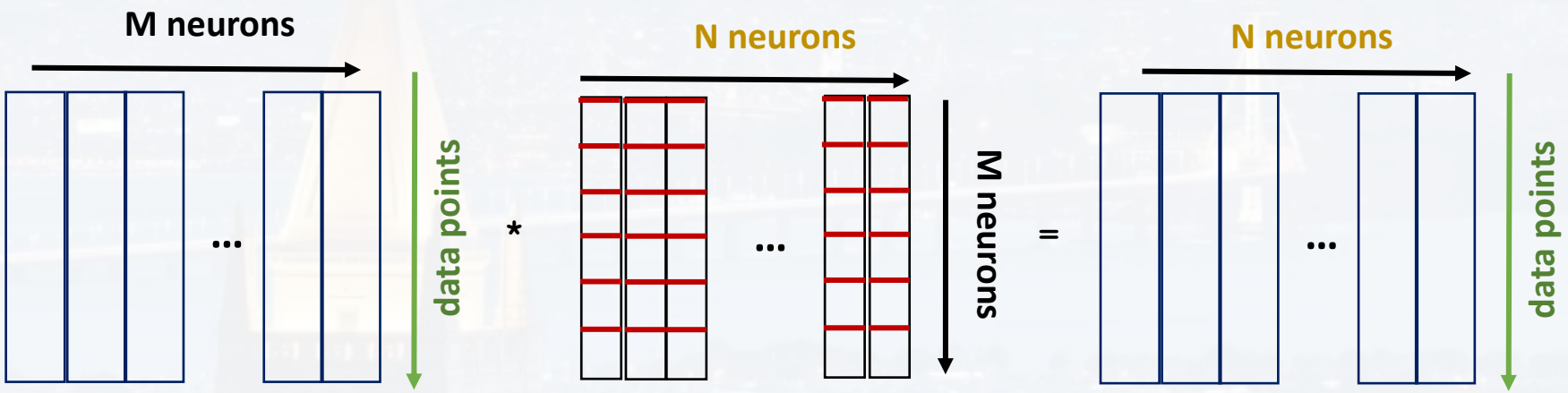


hidden layer

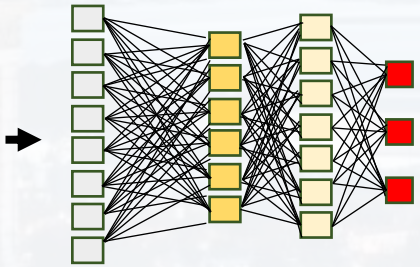
Introduction to Artificial Neural Networks: Multiple Dense Layers



input layer

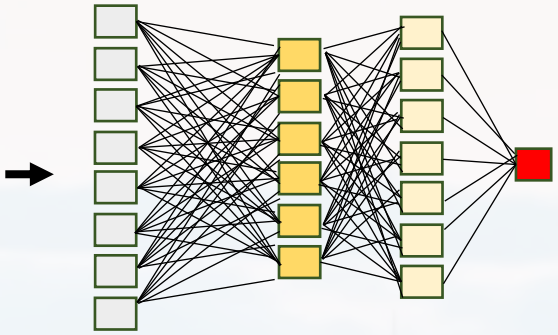


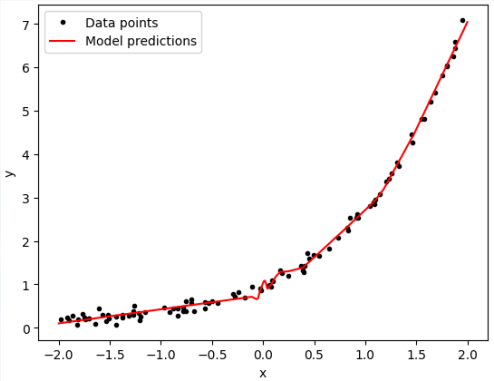
hidden layer



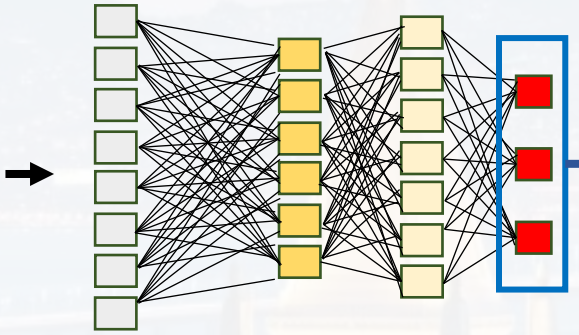
having many connected layers
+ activation function
→ ANN can solve highly non-linear problems!

Introduction to Artificial Neural Networks: Multiple Dense Layers





regression: one value per data point
→ **one single** output neuron



classification: probabilities per class per data point
→ **as many output neurons as classes**

softmax
(aka Boltzmann)

'setosa'	'versicolor'	'virginica'
1.0	0.0	0.0
0.0	1.0	0.0
0.0	0.4	0.6

probabilities

data points

Outline

- The Perceptron
- Gradient Descent & Backpropagation
- Multiple Dense Layers
- **Examples in Keras and PyTorch**
- Feature Selection & Regularization (again)
- Other Architectures

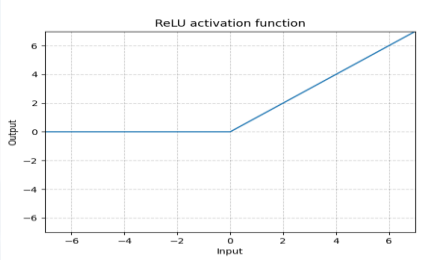
Introduction to Artificial Neural Networks: Examples in Keras and PyTorch

```
import tensorflow as tf
from tensorflow import keras
from tensorflow.keras import layers
```

minimal setup: input and out put layer

sequential: adding a stack of layers

input layer with subsequent ReLu




```
def My_Keras_Model(Nneuron, NClasses):

        return keras.Sequential([
                                 layers.Dense(Nneuron,  activation = 'relu'),
                                 layers.Dense(NClasses, activation = 'softmax')
                                ])
```

output layer with subsequent softmax for classification

minimal setup: input and out put layer

```
def My_Keras_Model(Nneuron, NClasses):

        return keras.Sequential([
                                 layers.Dense(Nneuron,  activation = 'relu'),
                                 layers.Dense(NClasses, activation = 'softmax')
                                ])
```

initializing the model

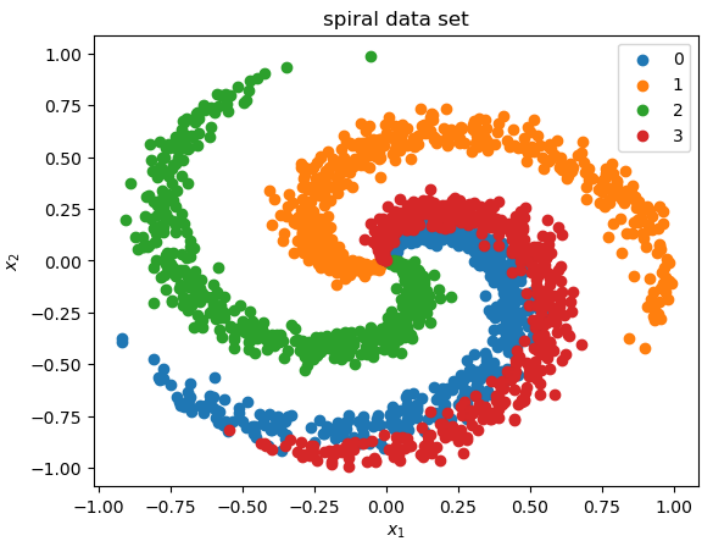
```
keras_model = My_Keras_Model(Nneuron, NClasses)
```

```
keras_model.compile(optimizer = 'adam', loss = 'categorical_crossentropy')
```

adding an optimizer and a loss function

goal: training the ANN to learn the labels

minimal setup: input and out put layer



four labels
two features

→ `Nclasses = 4`

training iterations (gradient descent)

```
Nepochs = 300
```

```
history = keras_model.fit(X_trainS, Y_train_oh, epochs           = Nepochs,
                                                validation_split = 0.2)
```

saving loss as function of epochs

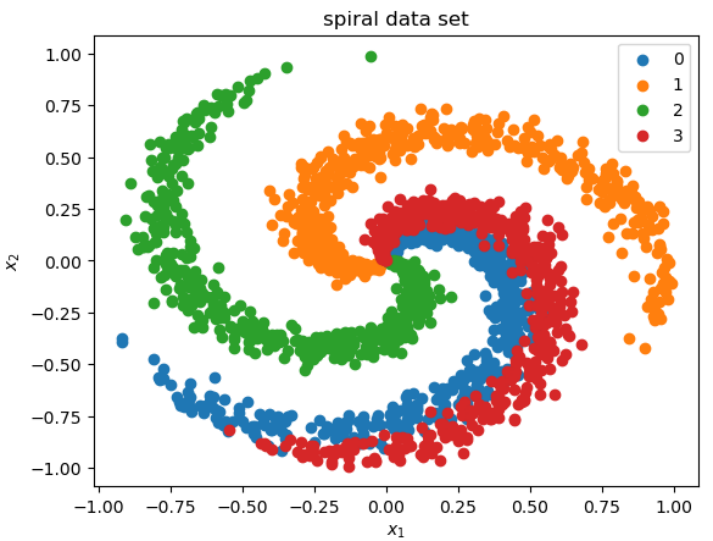
target: one-hot encoded classes

validation while training

Introduction to Artificial Neural Networks: Examples in Keras and PyTorch

goal: training the ANN to learn the labels

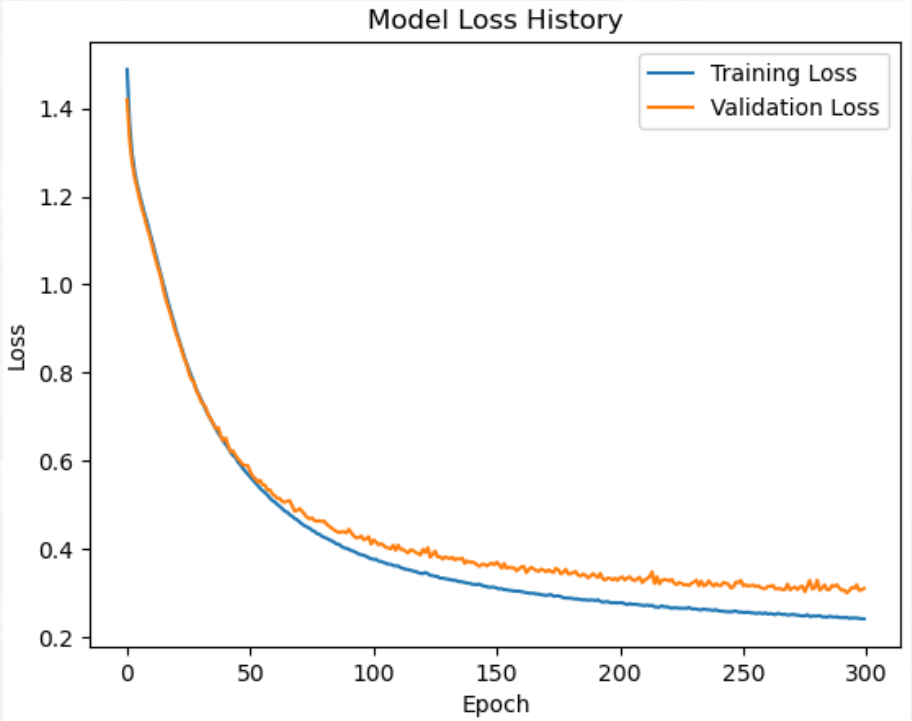
minimal setup: input and out put layer



four labels → `Nclasses = 4`

fwo features

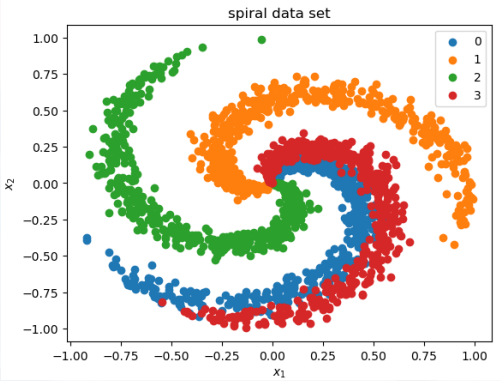
see `ANN_Keras.ipynb`



Introduction to Artificial Neural Networks: Examples in Keras and PyTorch

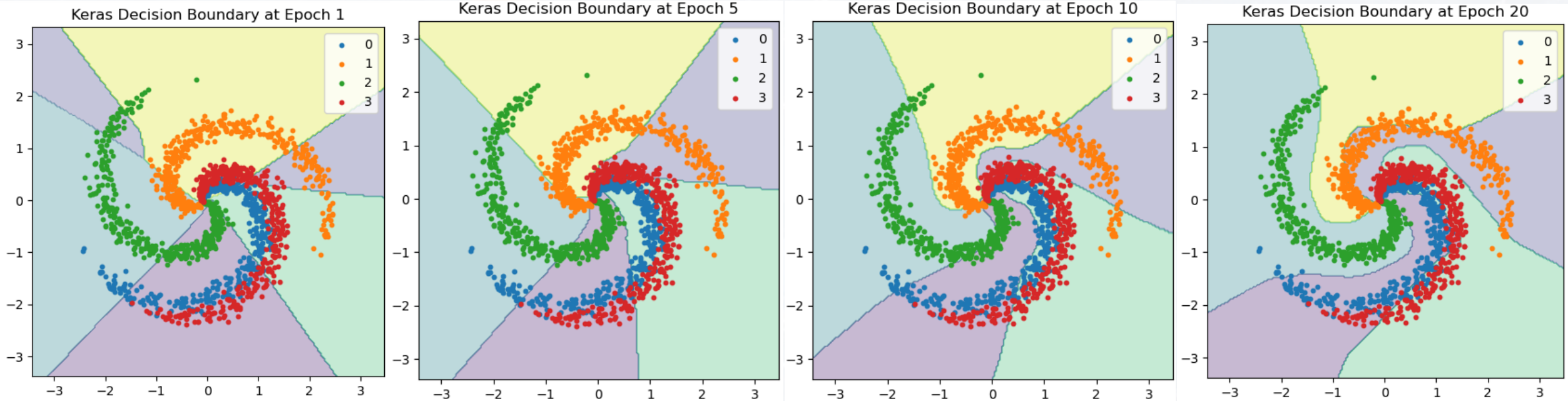
goal: training the ANN to learn the labels

minimal setup: input and out put layer



see `ANN_Keras.ipynb`

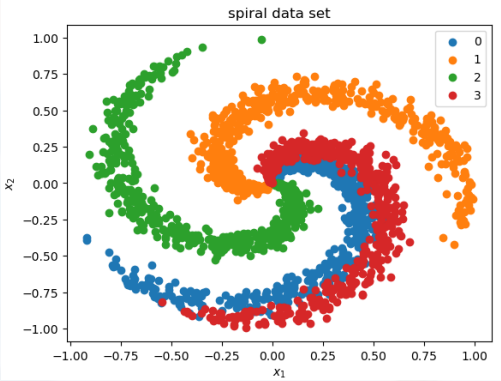
the learning process:



Introduction to Artificial Neural Networks: Examples in Keras and PyTorch

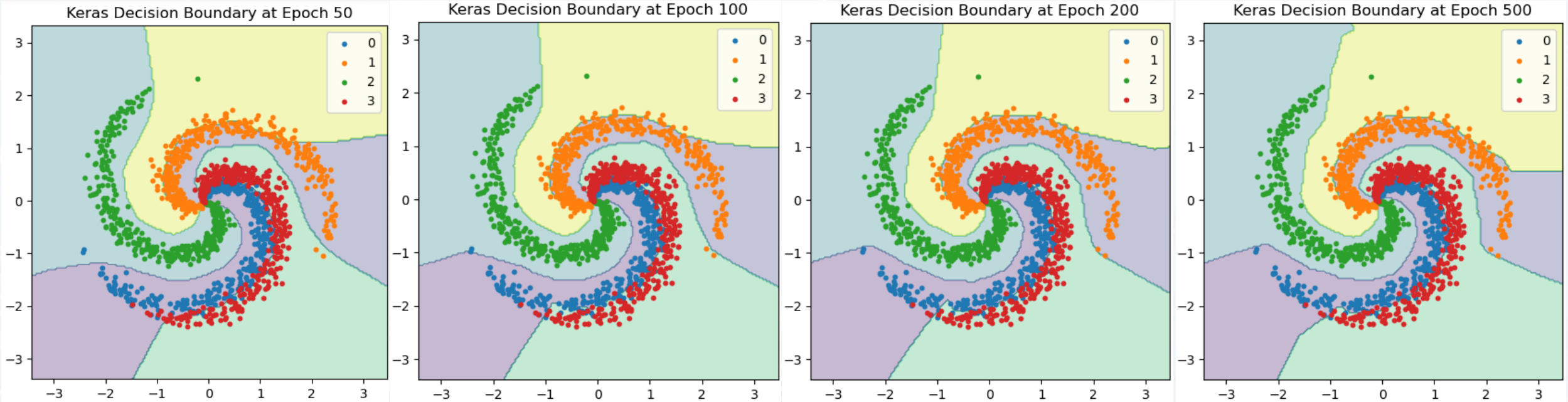
goal: training the ANN to learn the labels

minimal setup: input and out put layer



see `ANN_Keras.ipynb`

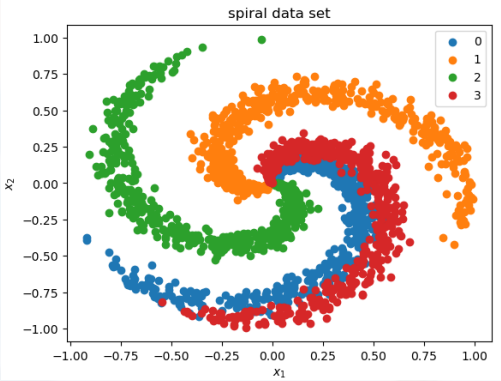
the learning process:



Introduction to Artificial Neural Networks: Examples in Keras and PyTorch

goal: training the ANN to learn the labels

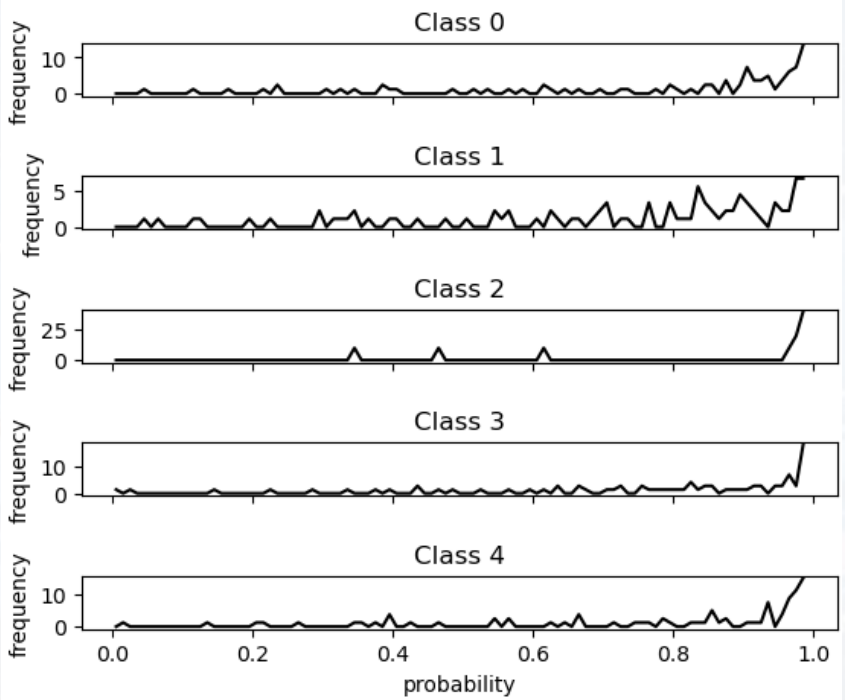
minimal setup: input and out put layer

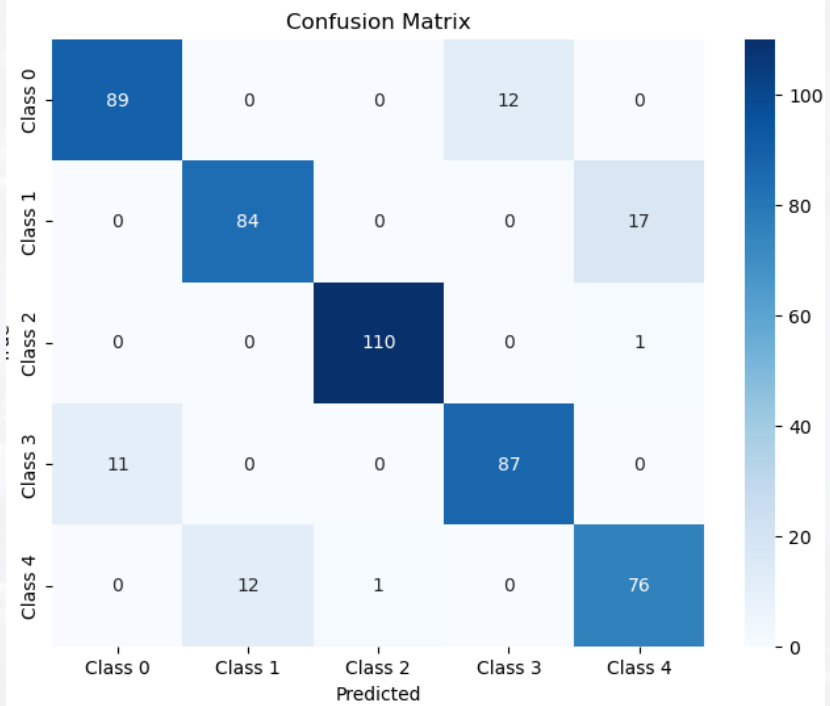


see `ANN_Keras.ipynb`

evaluation:

cross entropy:





now: same with PyTorch

```
import torch
import torch.nn as nn
import torch.optim as optim
import torch.nn.functional as nnf
```

for ANN methods like sequential, ReLU etc

for calling optimizers

needed later for calculating probabilities

defining a `class` is more conveinient

```
class My_Torch_Model(nn.Module):

    def __init__(self, NClasses, Nfeatures, Nneuron: int = 32):

        super().__init__()
        self.net =    nn.Sequential( nn.Linear(Nfeatures, Nneuron),
                                     nn.ReLU(),
                                     nn.Linear(Nneuron, NClasses))
```

input layer: `Linear` is equivalent to `Dense` in Keras

PyTorch requires explicit forward part

```
def forward(self, x):
    return self.net(x)
```

note: PyTorch adds `softmax` automatically if we define cross entropy as loss later

number of features

PyTorch models can run on different devices such as CPU and GPU

```
model          = My_Torch_Model(NClasses, X.shape[1], Nneuron).to(device)
```

```
leaning_rate = 0.01
```

```
optimizer     = optim.Adam(model.parameters(), lr = leaning_rate)

criterion     = nn.CrossEntropyLoss()
```

adding an optimizer and a loss function

running the training loop in PyTorch

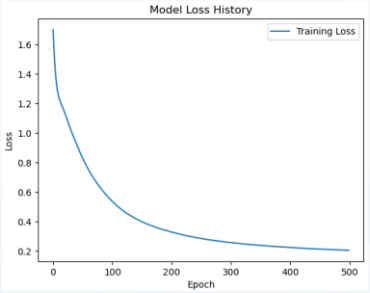
```
for e in range(Epochs):

        model.train()

        optimizer.zero_grad()

        out  = model(X_train_t)
        loss = criterion(out, Y_train_t)

        loss.backward()

        optimizer.step()

        print(f"current epoch: {e + 1}\ncurrent loss: {loss: .2f}")
```

PyTorch needs you to explicitly clear the gradient values from the previous training iteration

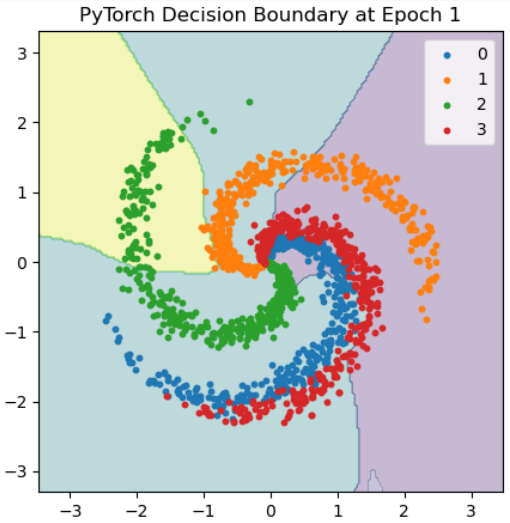
calculating the loss

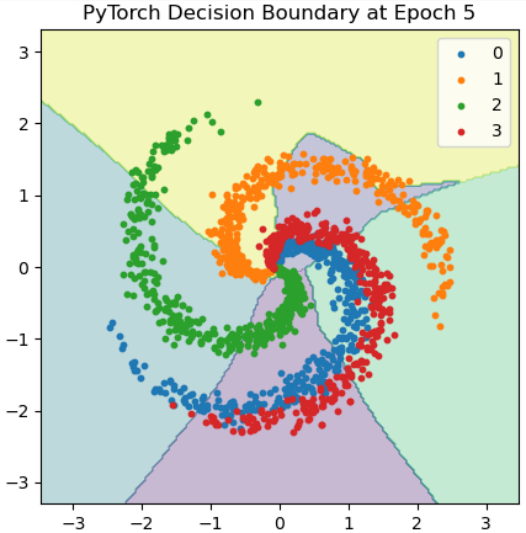
the loss is the outermost derivative for back propagation

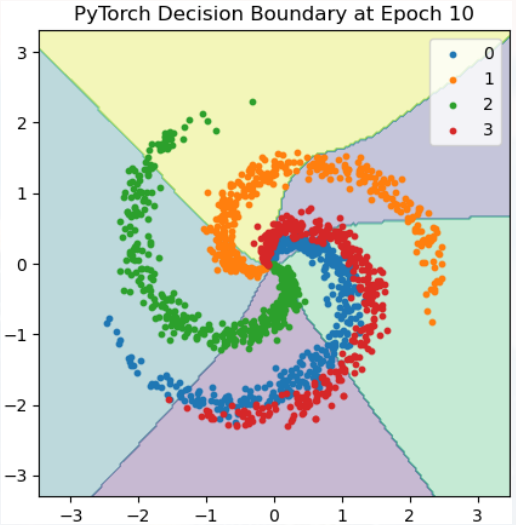
ready for the next training iteration

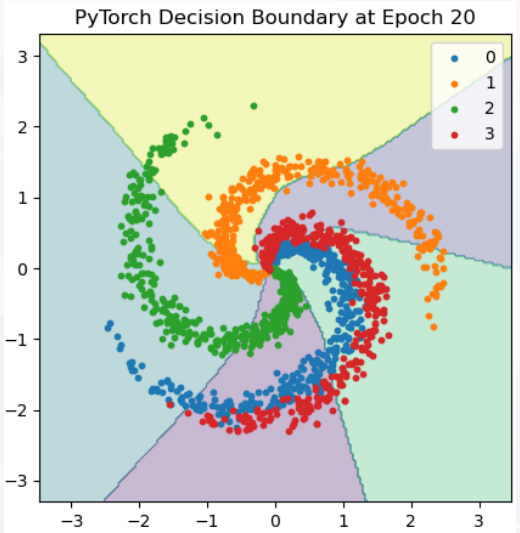


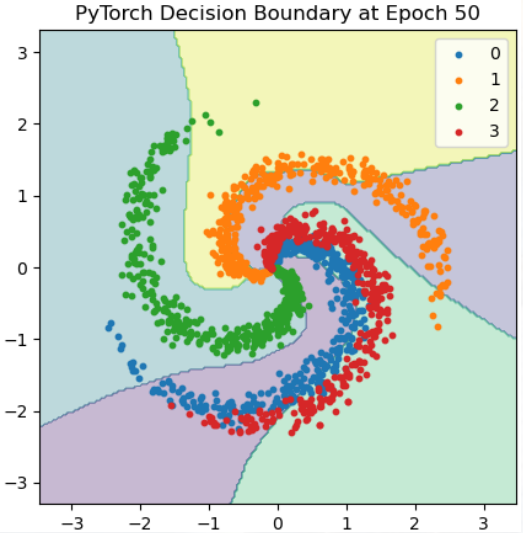
even on CPU: PyTorch is **much faster** than Keras

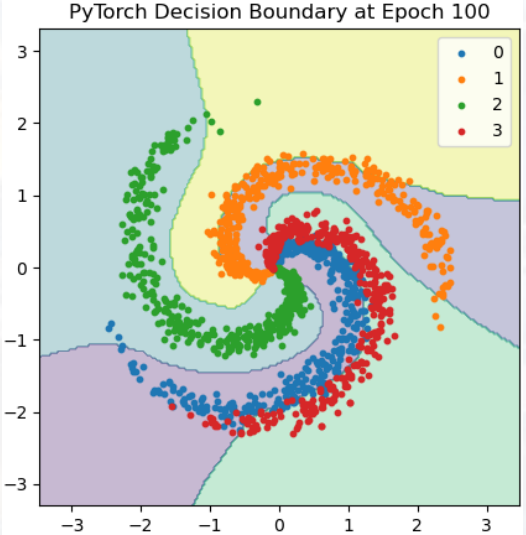


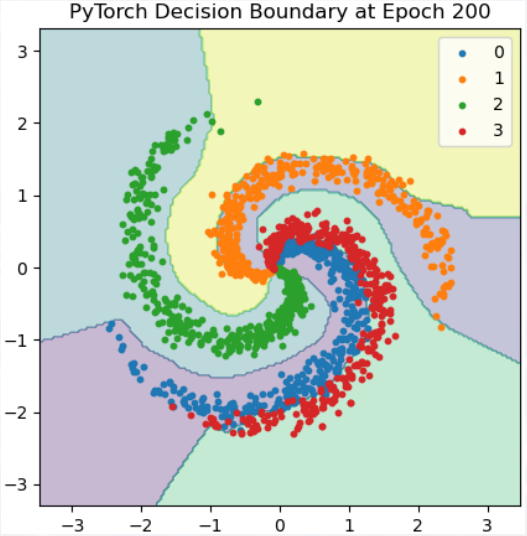


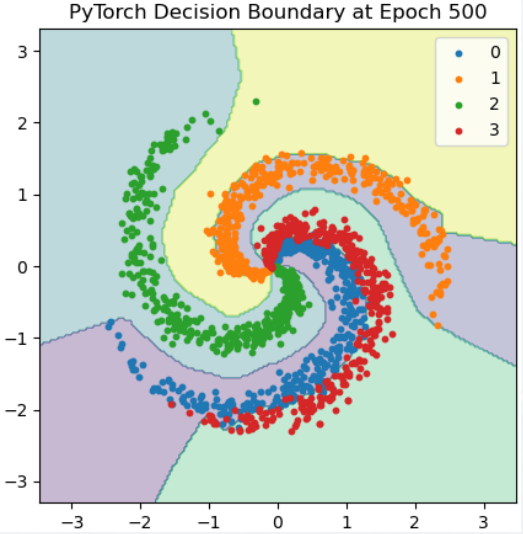










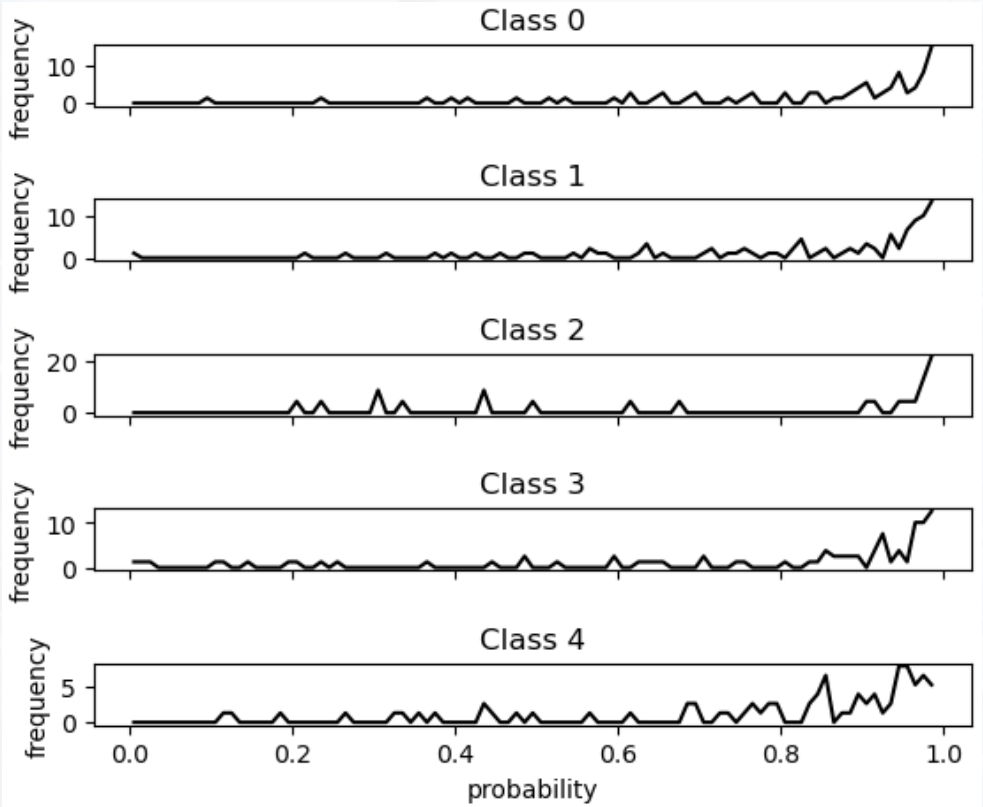


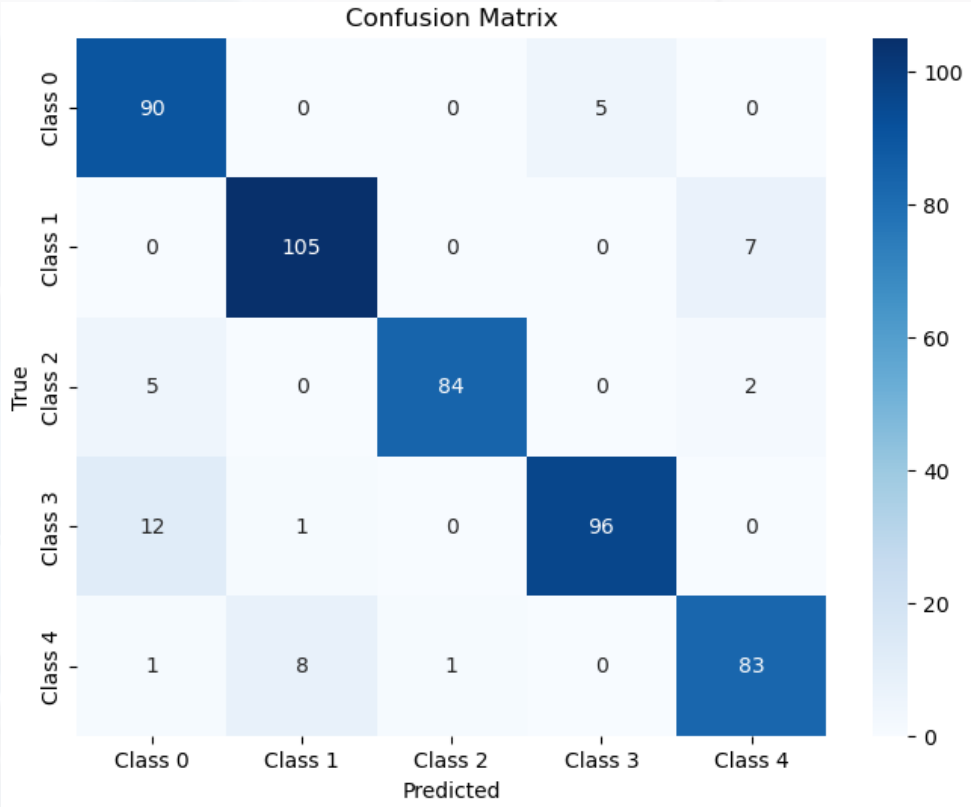
evaluation:

```
logits  = model(X_test_t)

PredY   = torch.argmax(logits, dim = 1).numpy()

P       = nnf.softmax(logits, dim = 1).detach().numpy()
```

PyTorch does not return probabilities automatically → softmax

see ANN_PyTorch.ipynb





Outline

- The Perceptron
- Gradient Descent & Backpropagation
- Multiple Dense Layers
- Examples in Keras and PyTorch
- **Feature Selection & Regularization** (again)
- Other Architectures

Introduction to Artificial Neural Networks: Feature Selection & Regularization

We can now revisit the diabetes dataset

recall: we didn't need all features

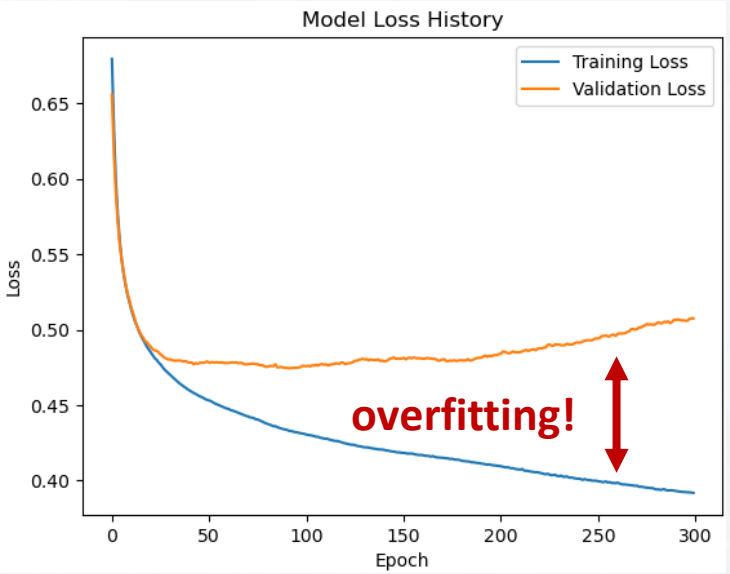
	Pregnancies	Glucose	BloodPressure	SkinThickness	Insulin	BMI	DiabetesPedigreeFunction	Age	label
0	6	148.0	72.0	35.00	184.42	33.6	0.63	50	diabetes
1	1	85.0	66.0	29.00	75.91	26.6	0.35	31	healthy
2	8	183.0	64.0	31.34	155.87	23.3	0.67	32	diabetes
3	1	89.0	66.0	23.00	94.00	28.1	0.17	21	healthy
4	0	137.0	40.0	35.00	168.00	43.1	2.29	33	diabetes

- best model via L1 and L2
- supported by stepwise dropping features
- max accuracy 78%

```
alpha_1 = 0.085
alpha_2 = 0.85
```

	coef	std err	z	P>|z|	[0.025	0.975]
const	-0.8517	0.122	-6.966	0.000	-1.091	-0.612
Glucose	1.1150	0.133	8.398	0.000	0.855	1.375
BMI	0.7348	0.128	5.719	0.000	0.483	0.987

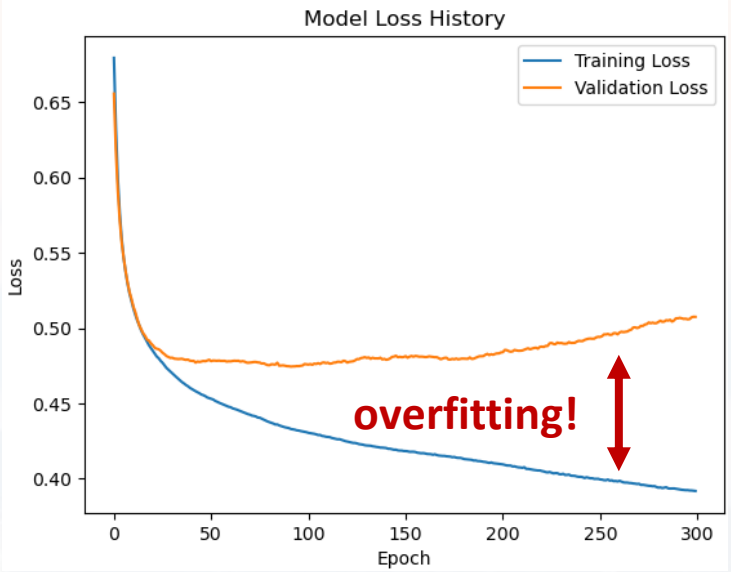
running the Keras model with just **eight neurons** for the input layer



note: - L1 is applicable to ANNs too, but that relates to the **weights**, not the features directly!

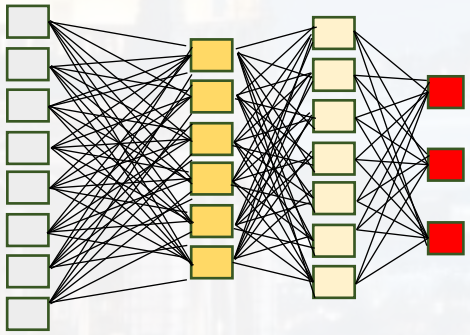
$$net = \sum_i I_i \cdot w_i + b$$

- we have no p-values



drop-out: - regularization (still relates to the weights!)

- each training step: a certain faction, say 20% (= **drop out rate**) of randomly picked neurons "drop-out", their output is set to zero

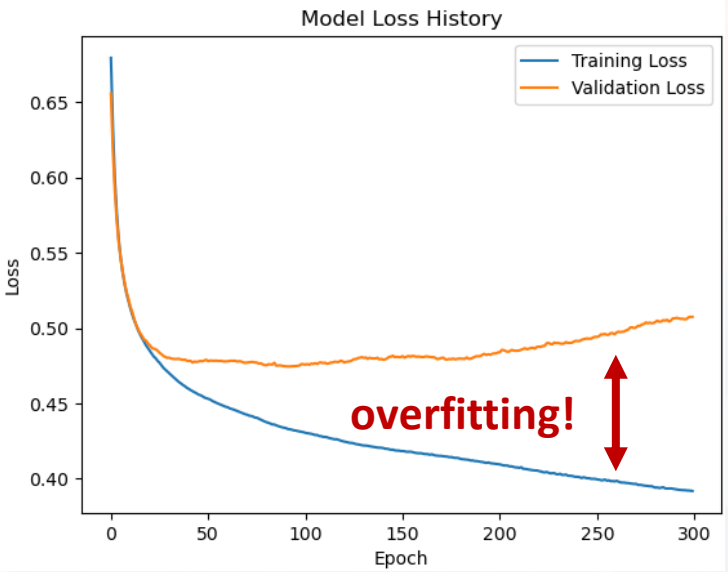


note:

- L1 is applicable to ANNs too, but that relates to the **weights**, not the features directly!

- we have no p-values

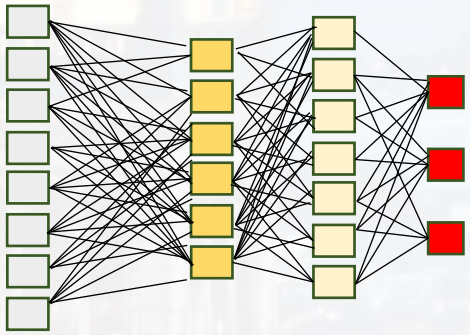
$$net = \sum_i I_i \cdot w_i + b$$



drop-out:

- regularization (still relates to the weights!)

- each training step: a certain faction, say 20% (= **drop out rate**) of randomly picked neurons "drop-out", their output is set to zero

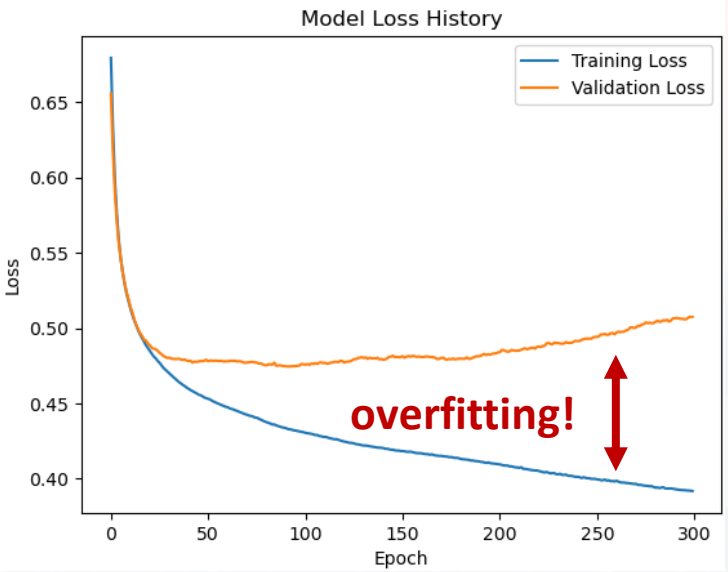


note:

- L1 is applicable to ANNs too, but that relates to the **weights**, not the features directly!

- we have no p-values

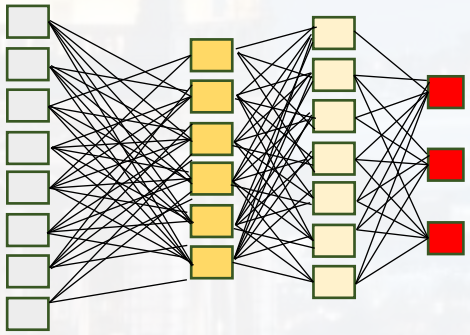
$$net = \sum_i I_i \cdot w_i + b$$



drop-out:

- regularization (still relates to the weights!)

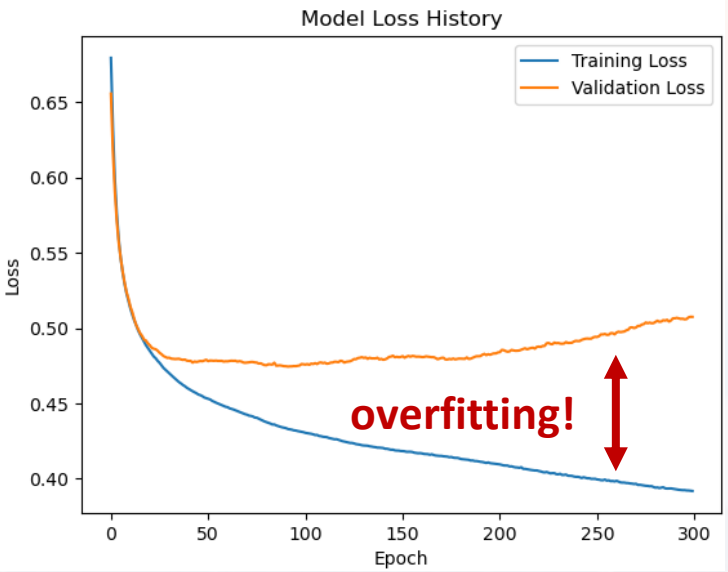
- each training step: a certain faction, say 20% (= **drop out rate**) of randomly picked neurons "drop-out", their output is set to zero



note: - L1 is applicable to ANNs too, but that relates to the **weights**, not the features directly!

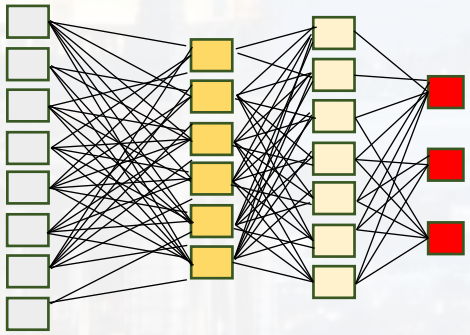
$$net = \sum_i I_i \cdot w_i + b$$

- we have no p-values



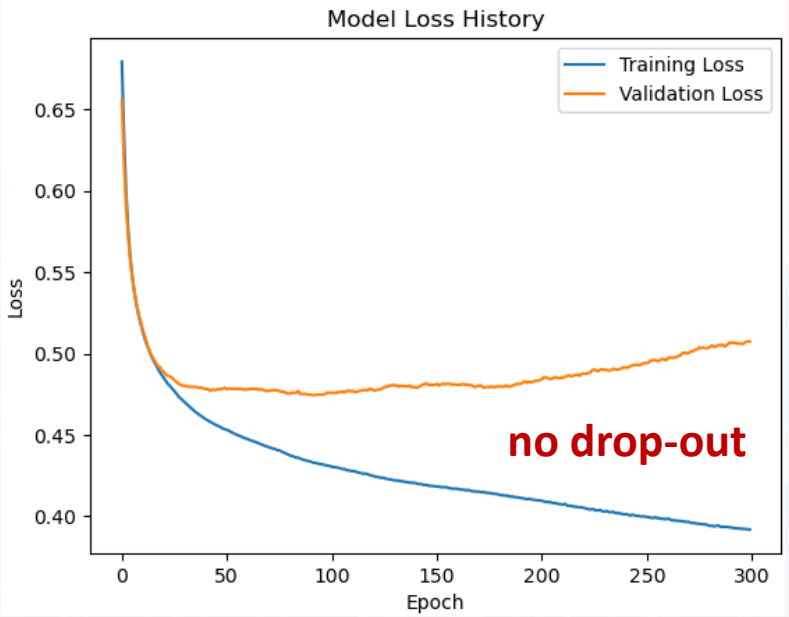
drop-out: - regularization (still relates to the weights!)

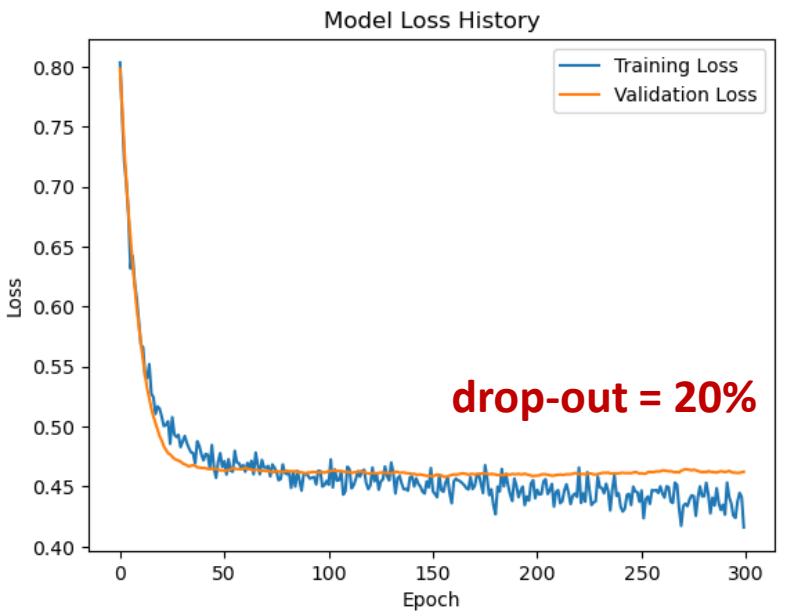
- each training step: a certain faction, say 20% (= **drop out rate**) of randomly picked neurons "drop-out", their output is set to zero



```
def My_Keras_Model(input_dim, Nneurons, NClasses, DArate = .8):

        model = keras.Sequential([
                                  layers.Input(shape = (input_dim,)),
                                  layers.Dense(Nneurons, activation = 'relu'),
                                  layers.Dropout(DArate),
                                  layers.Dense(NClasses, activation = 'softmax')
                                        ])

        model.compile(optimizer = 'adam', loss = 'categorical_crossentropy',
                      metrics = ['accuracy'])

        return model
```

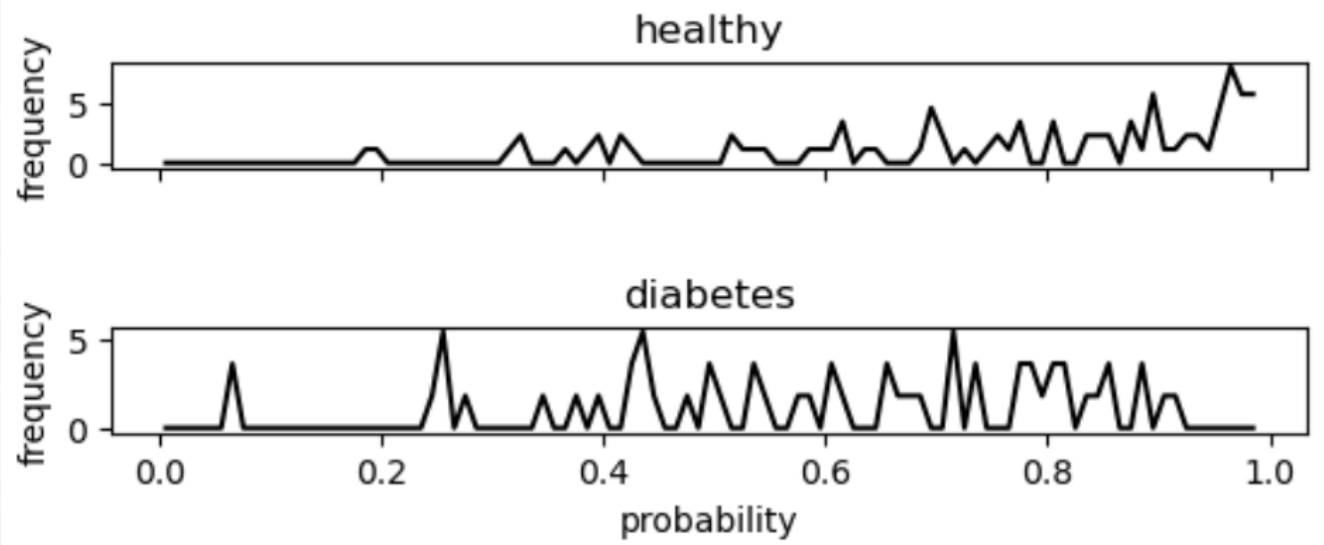
Introduction to Artificial Neural Networks: Feature Selection & Regularization

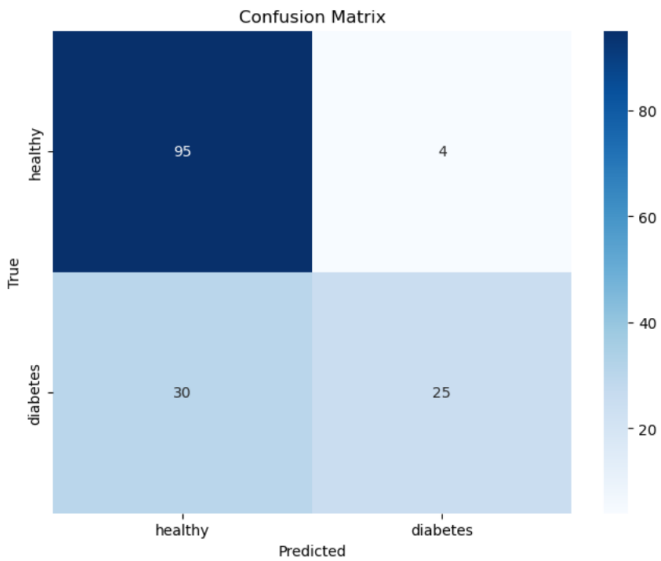





```
Accuracy =  79.87
```

the best result so far!





We solved the problem of overfitting, but still don't know which features are significant!

```
Accuracy =  79.87
```

→ **R**ecursive **F**eature **E**limination

A)

1) remove one feature and run model
2) calculate test accuracy and store the value
3) put feature back
4) got to 1)

B) once done: remove the feature whose drop is associated with highest accuracy (= least predictive feature)

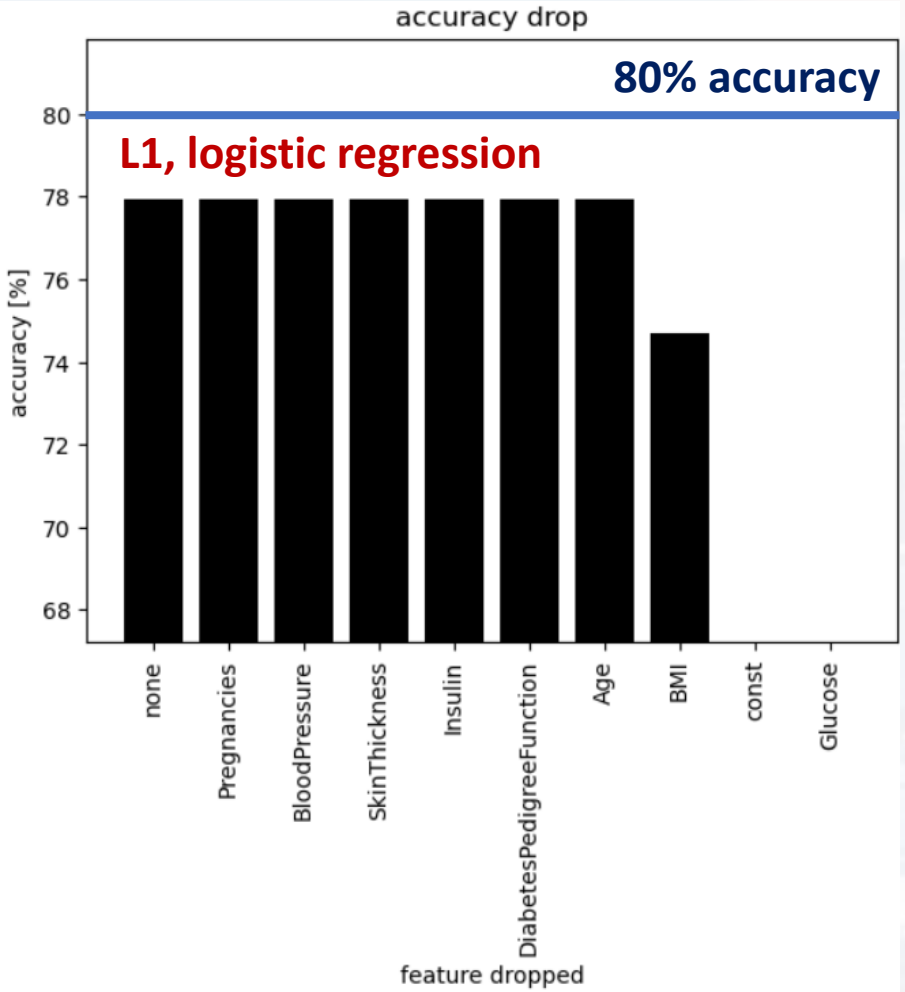
C) go to A) until only one feature is left

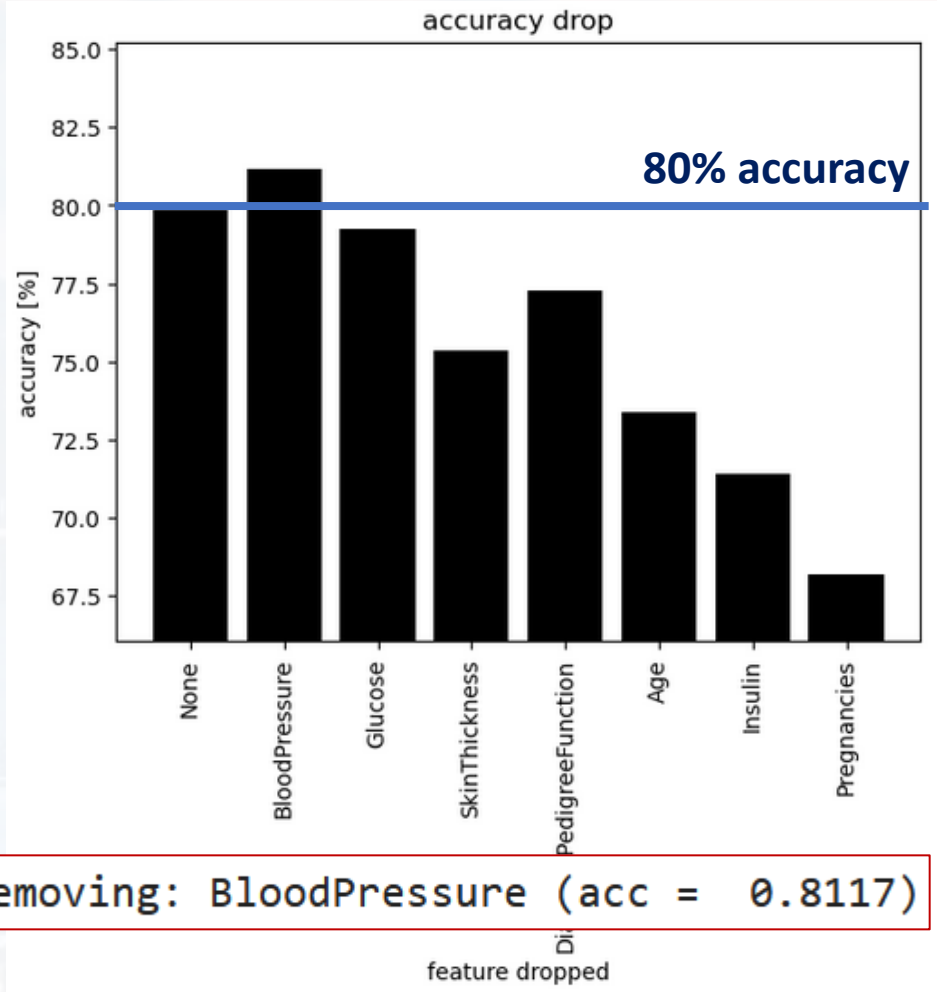
Ideally, you run each step with k-fold validation

Introduction to Artificial Neural Networks: Feature Selection & Regularization

see `FeatureSelection_ANN.ipynb`

Recursive Feature Elimination
ANN, drop-out 20%



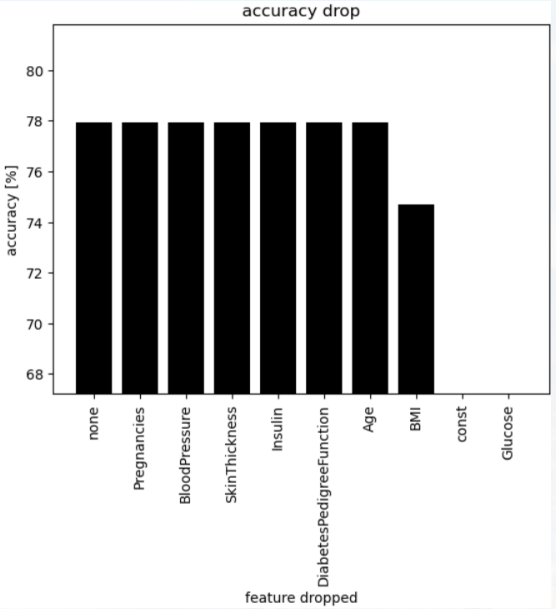



```
→ Removing: BloodPressure (acc =  0.8117)
```

Introduction to Artificial Neural Networks: Feature Selection & Regularization

see `FeatureSelection_ANN.ipynb`

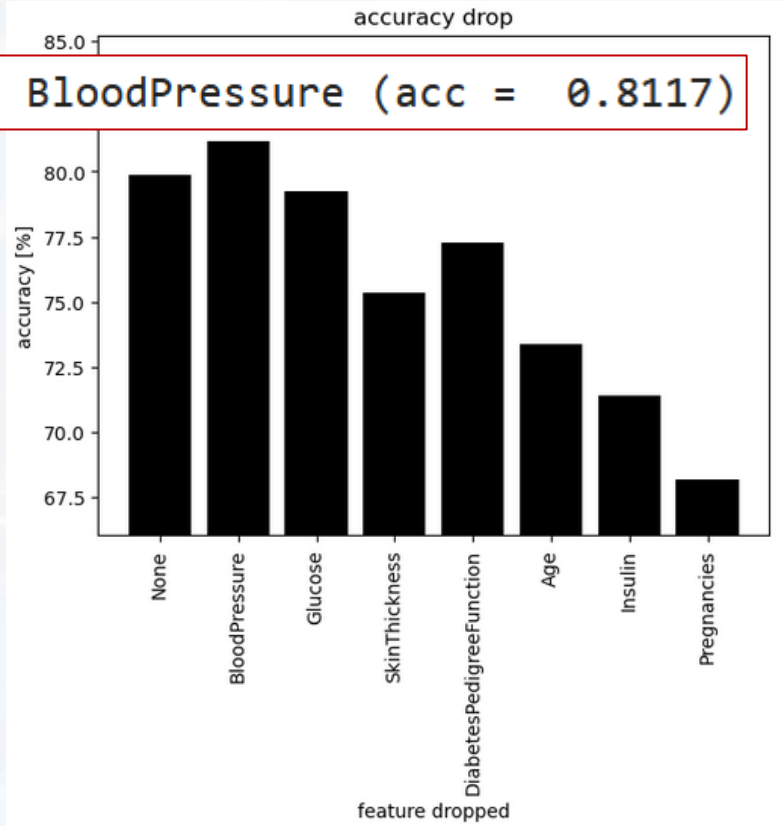
L1, logistic regression



78% accuracy

Recursive Feature Elimination
ANN, drop-out 20%

```
→ Removing: BloodPressure (acc =  0.8117)
```



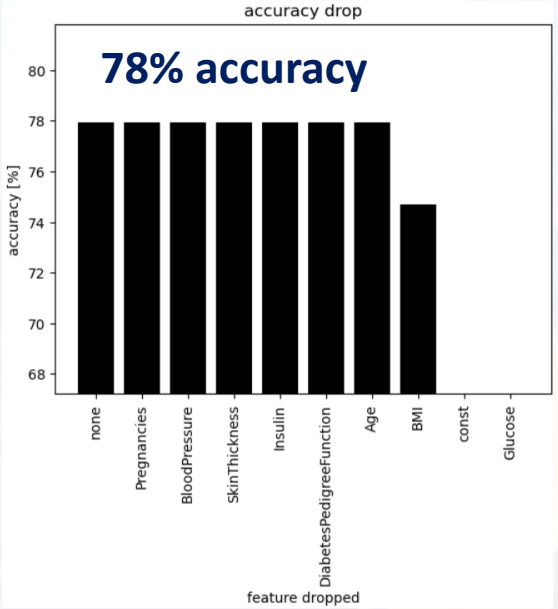
note: models are not 100% comparable:
logistic regression model includes a constant

RFE computational cost goes with $\frac{n^2}{2}$ if n = number of features
→ always run feature selection before if possible (PCA)

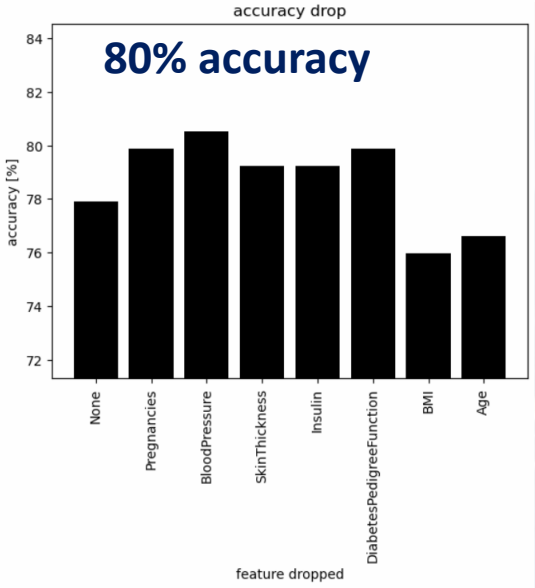
Introduction to Artificial Neural Networks: Feature Selection & Regularization

see `FeatureSelection_ANN.ipynb`

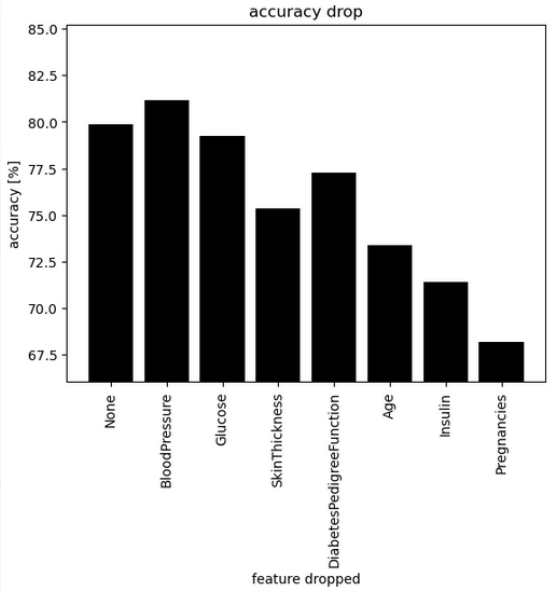
L1, logistic regression



Recursive Feature Elimination ANN, drop-out 20%, 250 epochs



Recursive Feature Elimination ANN, drop-out 20%, 100 epochs



note: models are not 100% comparable:
logistic regression model includes a constant

RFE computational cost goes with $\frac{n^2}{2}$ if n = number of features
→ always run feature selection before if possible (PCA)

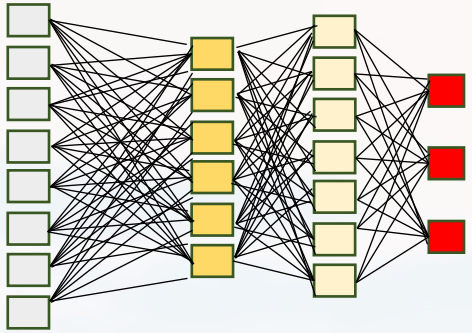
Outline

- The Perceptron
- Gradient Descent & Backpropagation
- Multiple Dense Layers
- Examples in Keras and PyTorch
- Feature Selection & Regularization (again)
- **Other Architectures**

Introduction to Artificial Neural Networks: Other Architectures

more details: 277B

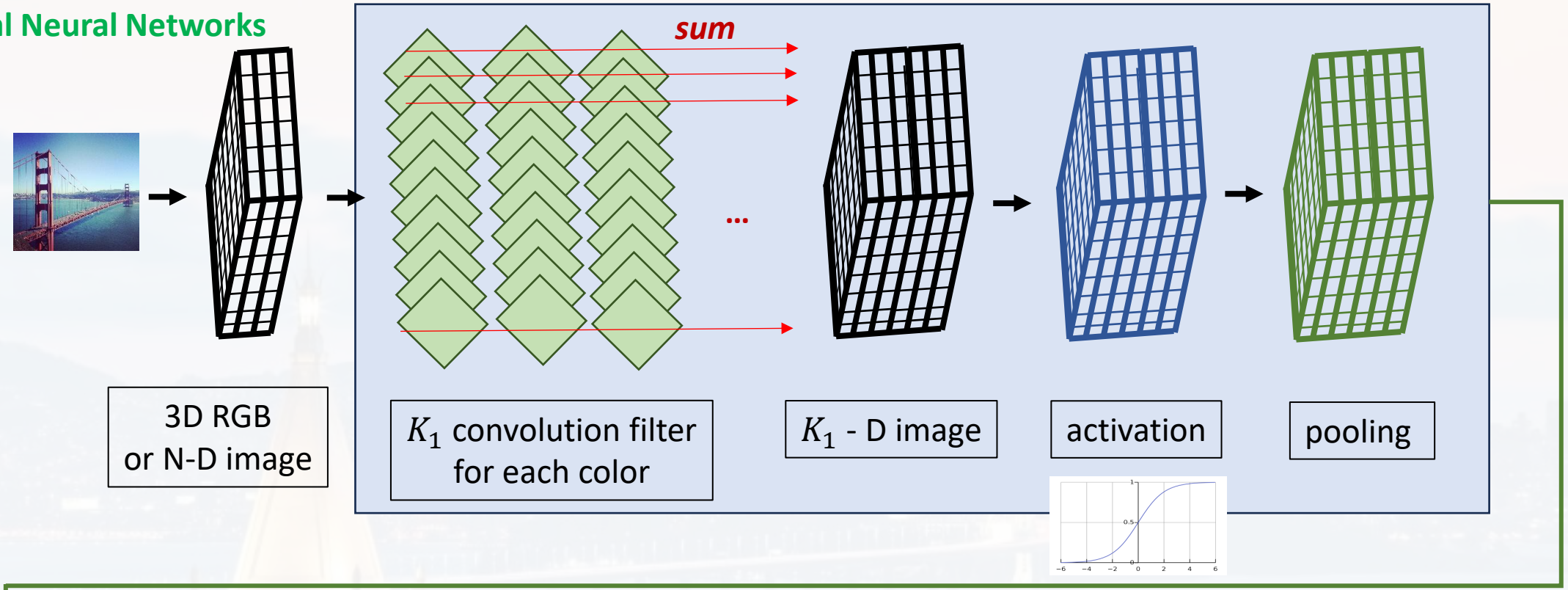
dense network

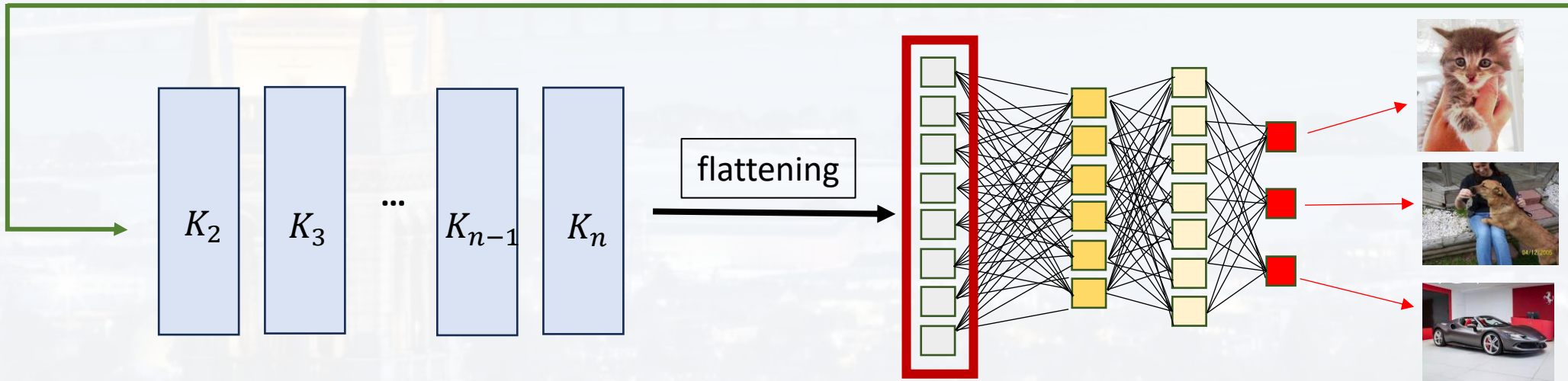


core structure of most deep learning architectures

Introduction to Artificial Neural Networks: Other Architectures

Convolutional Neural Networks

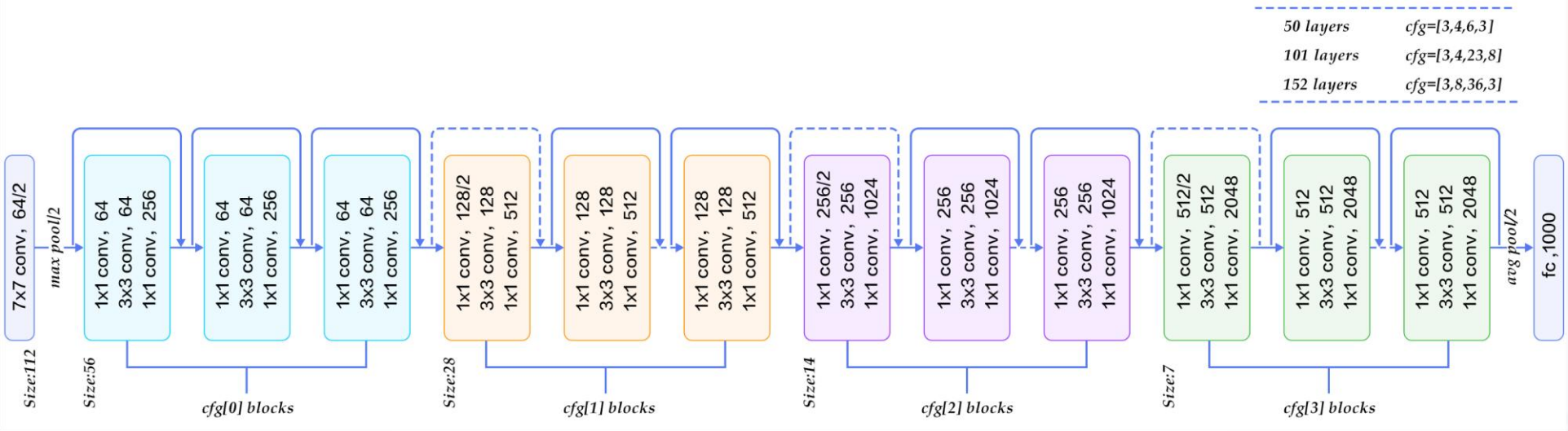


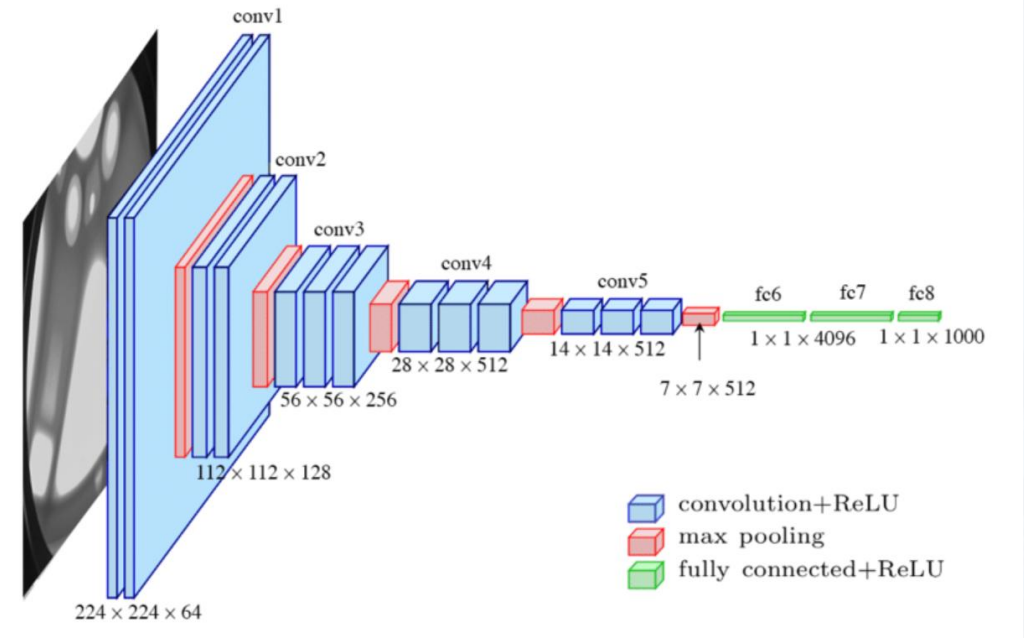


Introduction to Artificial Neural Networks: Other Architectures

Convolutional Neural Networks

more details: 277B

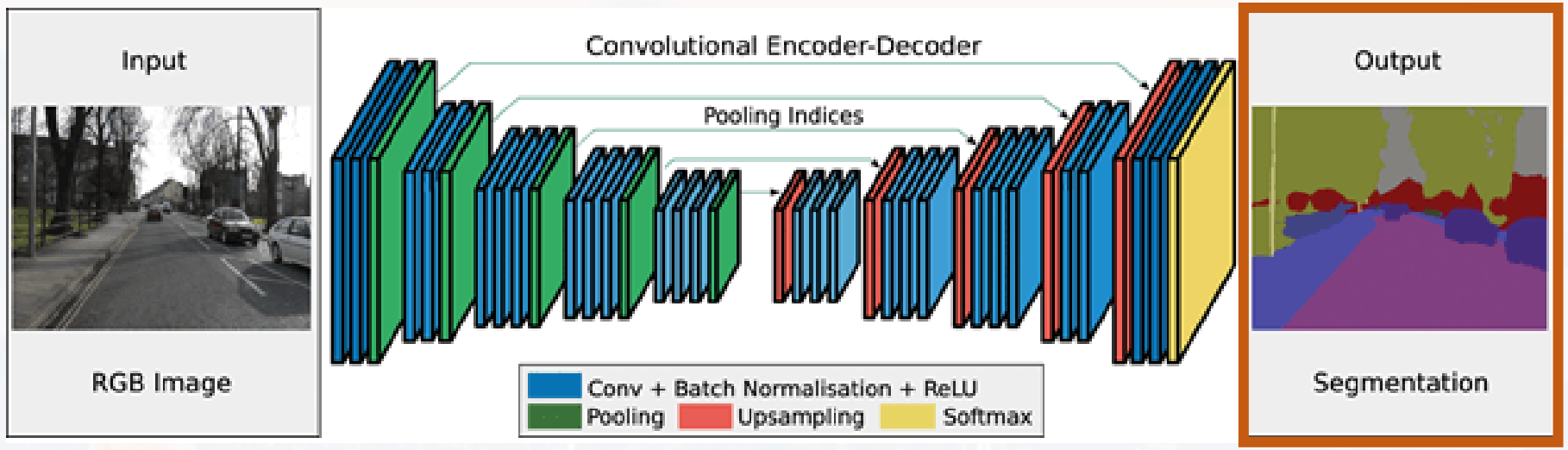




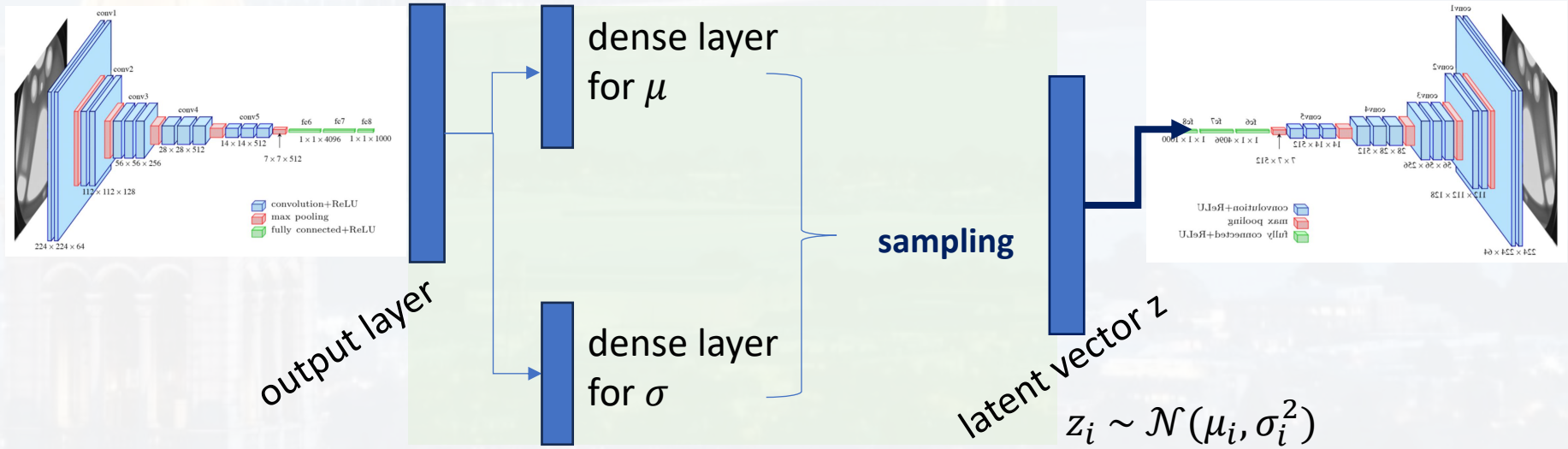
Convolutional Neural Networks

more details: 277B

segmentation



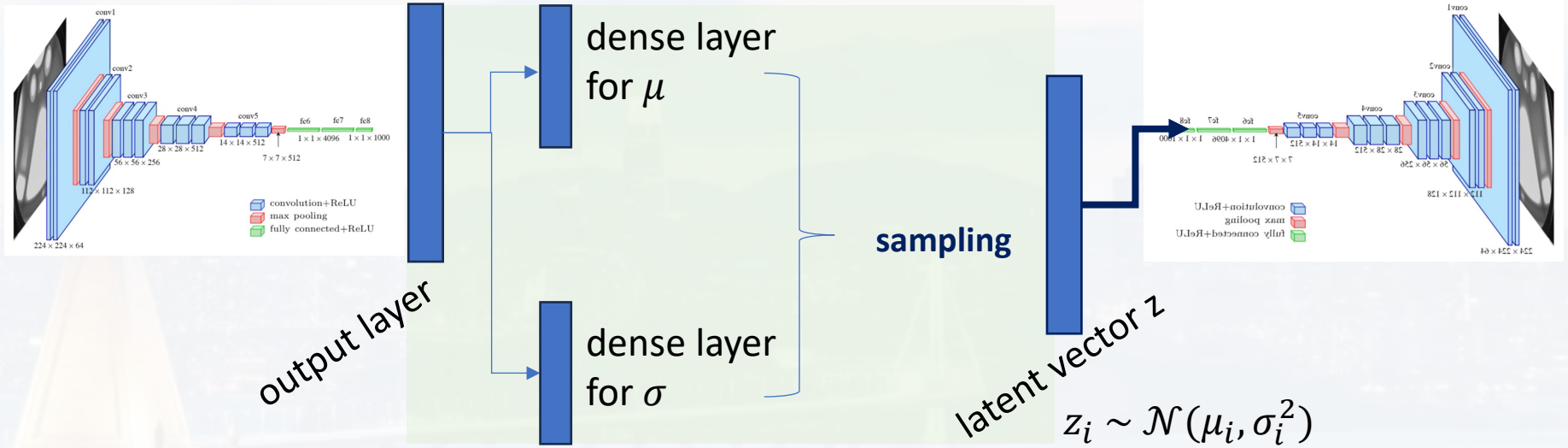
Variational AutoEncoder (VAE) image generation



Convolutional Neural Networks

more details: 277B

Variational AutoEncoder (VAE) image generation

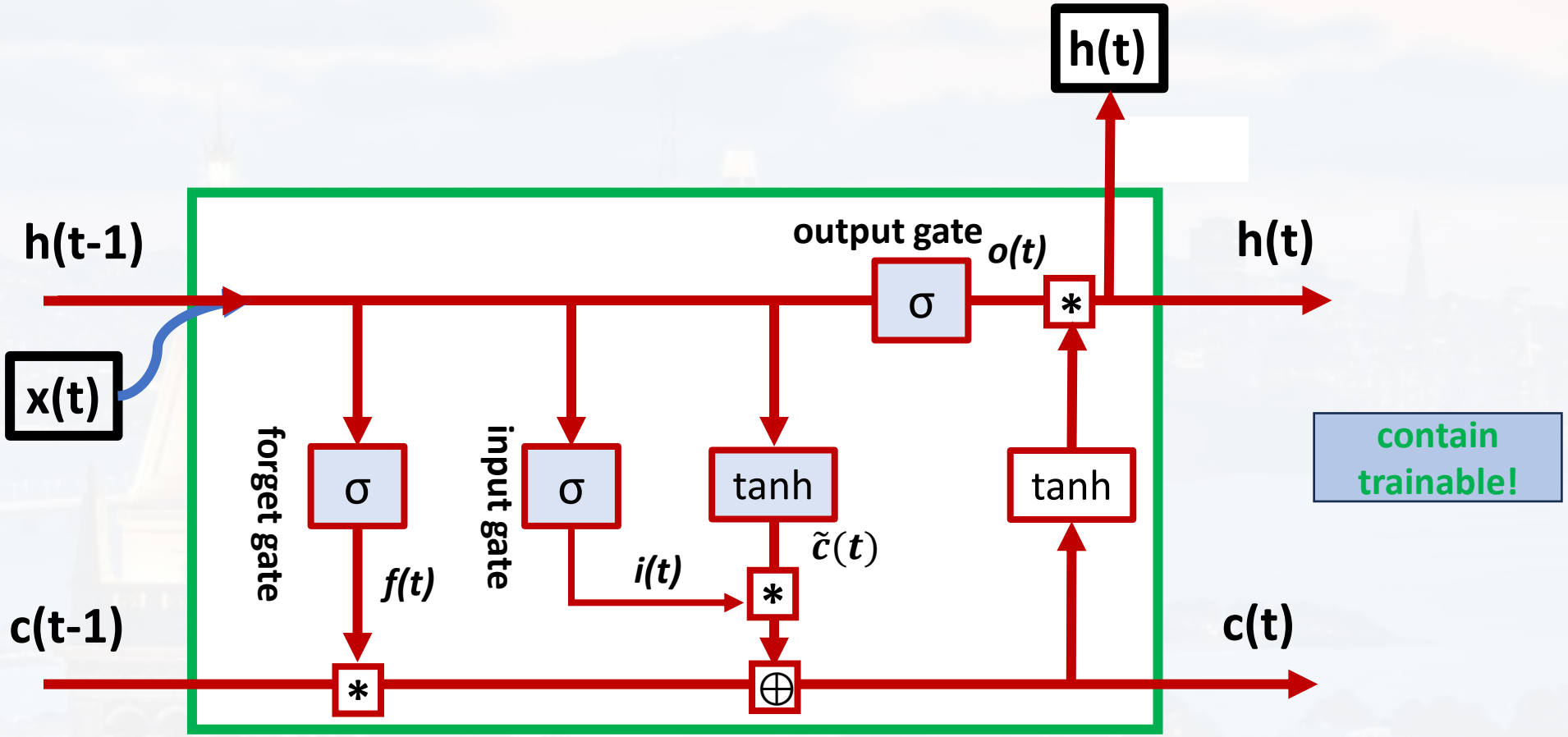


generated

Long- Short Term Memory (LSTM)

more details: 277B

time series



Introduction to Artificial Neural Networks: Other Architectures

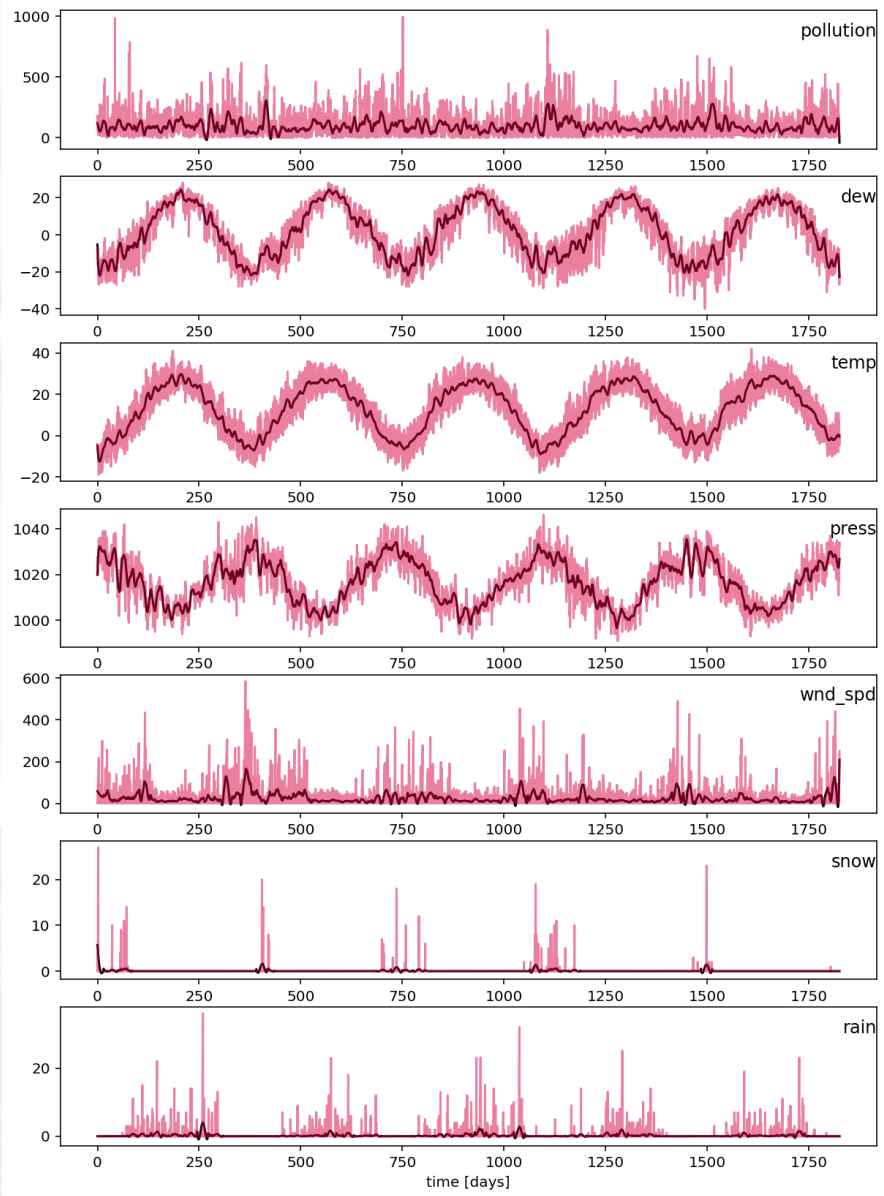
Long- Short Term Memory (LSTM)

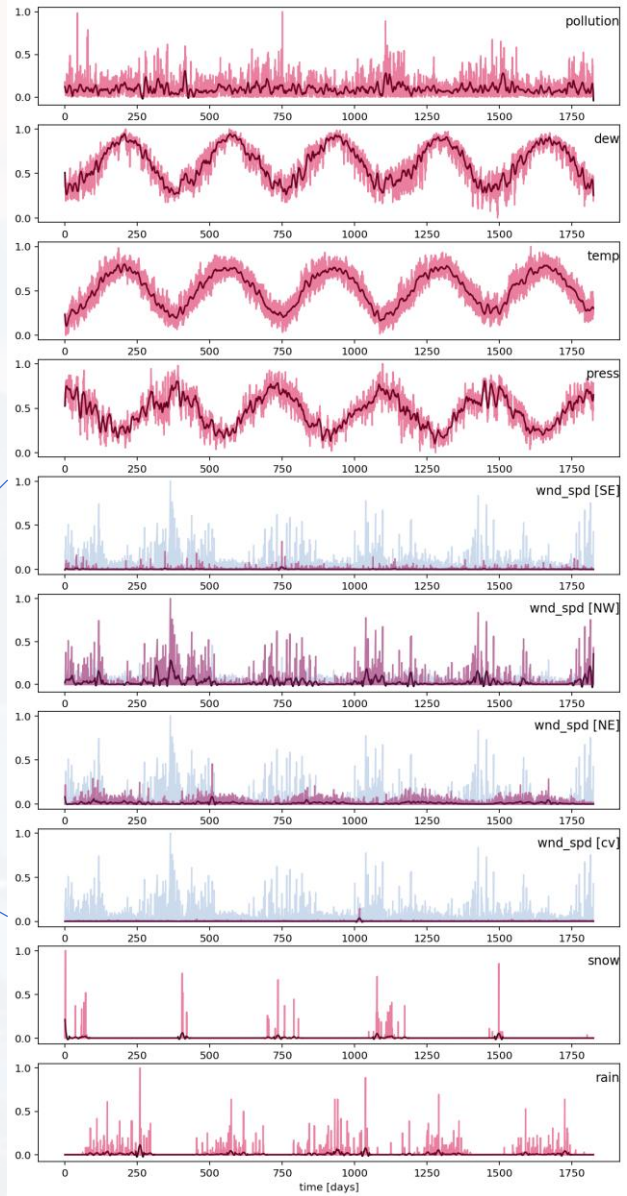
more details: 277B

time series

actual data

monthly moving average

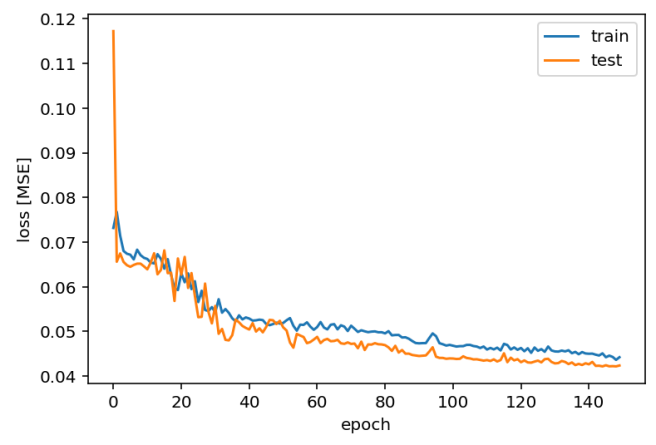


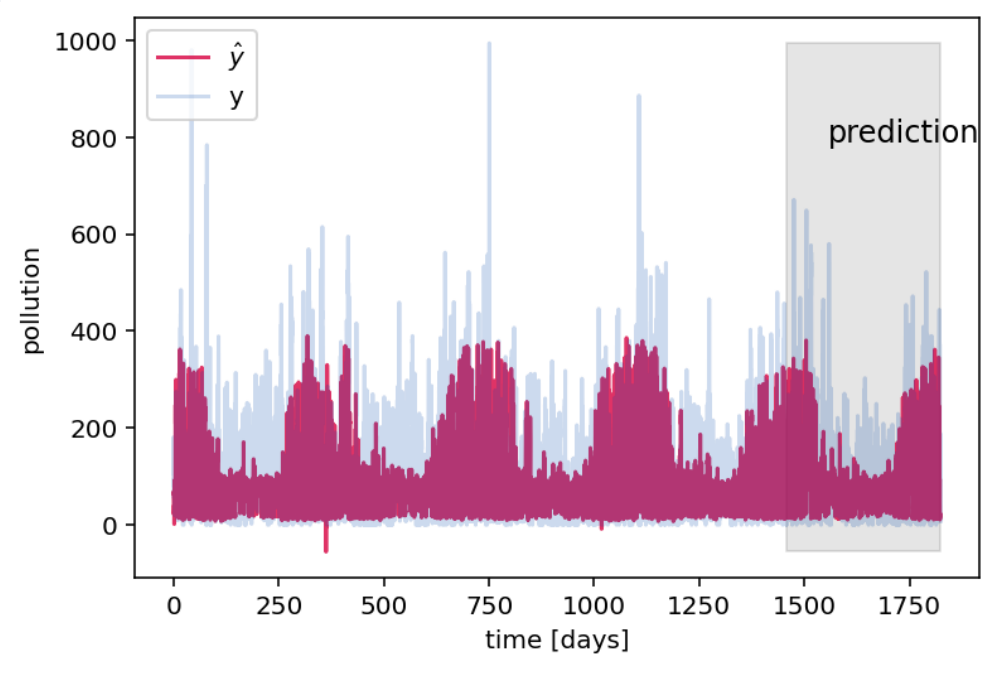


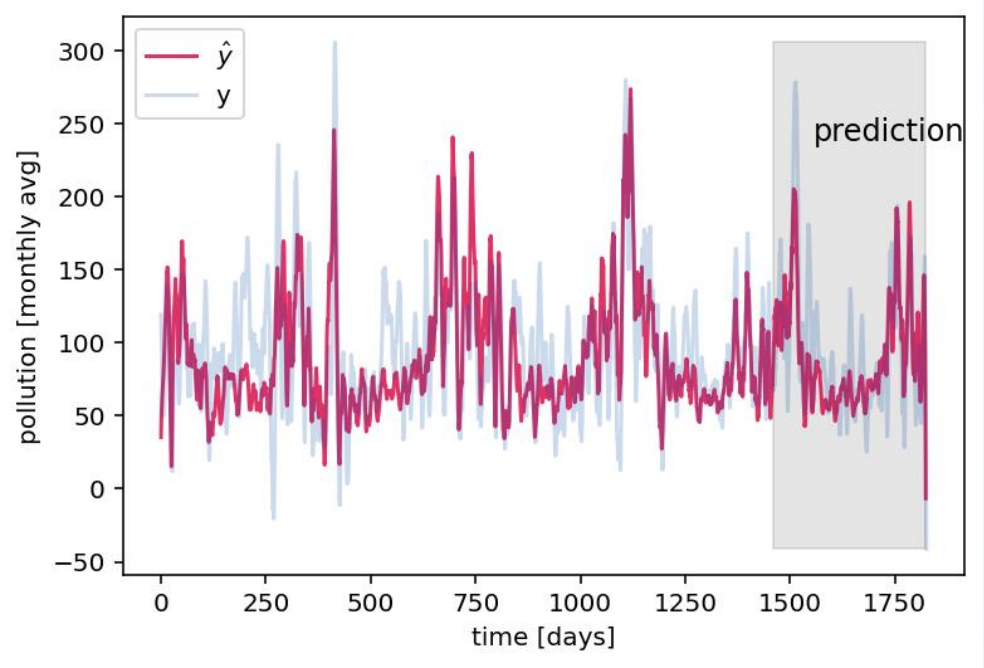
Long- Short Term Memory (LSTM)

more details: 277B

time series



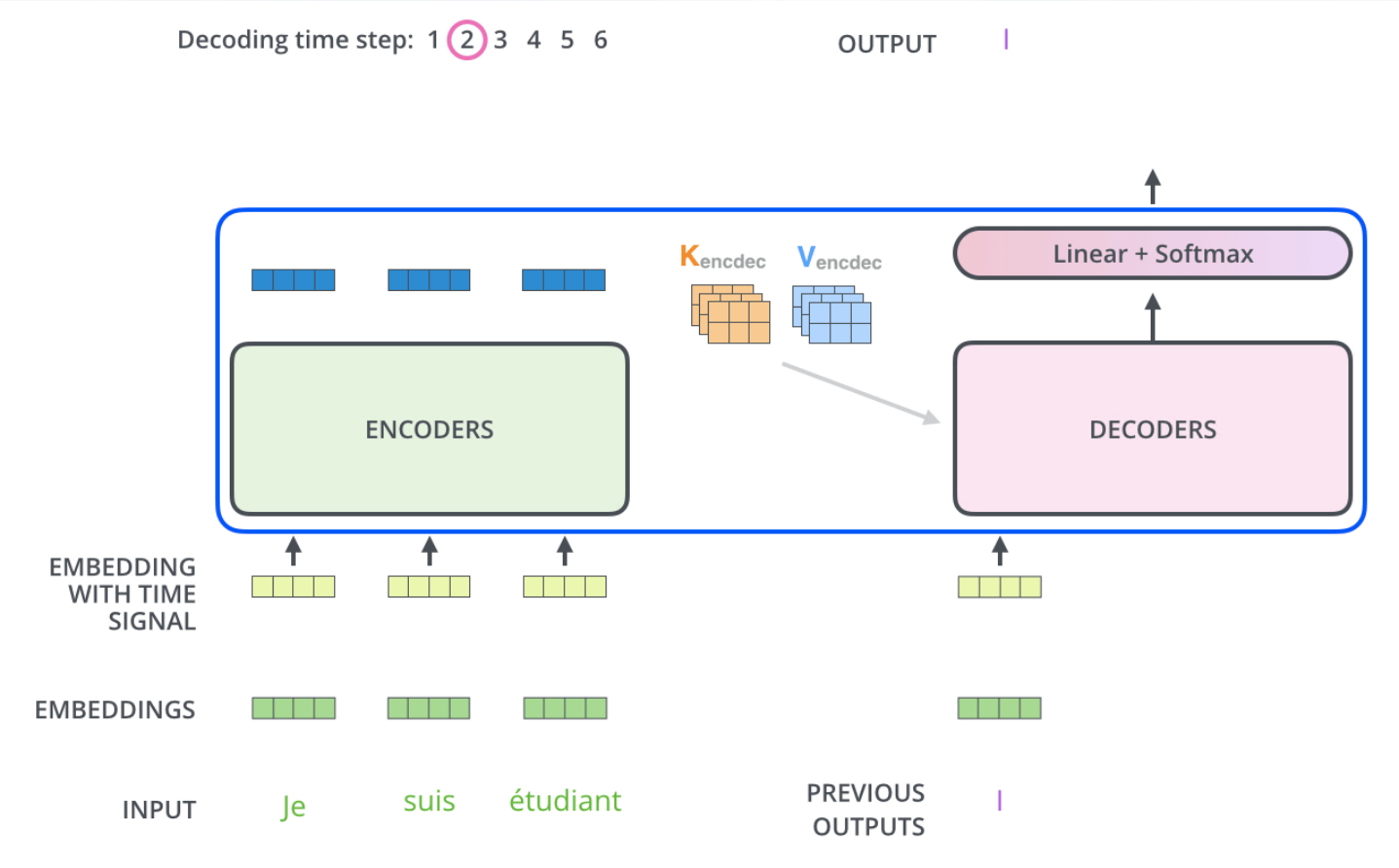


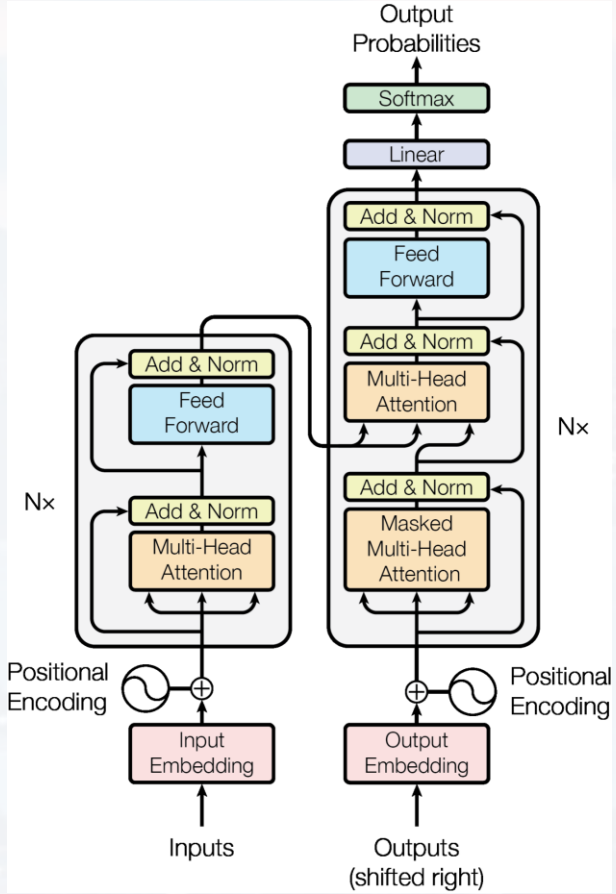


Transformer

more details: 277B

time series, DNA/RNA, natural language processing





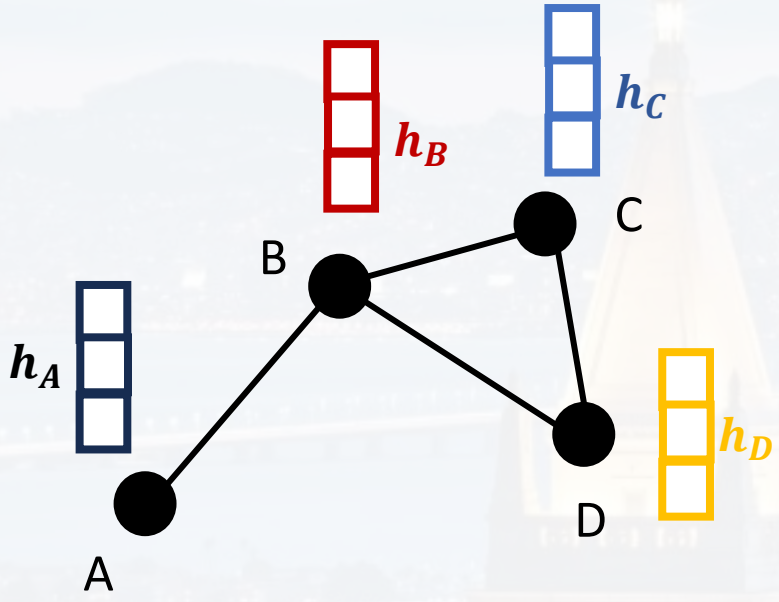
https://jalammar.github.io/illustrated-transformer/

Attention Is All You Need (Vaswani et al, 2017)

Graph Neural Networks

more details: 277B

3D structure of molecules (AlphaFold), project management, social networks, Google maps



all architectures can be combined:

graph + attention → generating new molecules (drug development, material science)

CNNs + transformer → vision transformer (image and video generation using diffusion mechanisms, object detection, segmentation, and medical imaging)

Thank you very much for your attention and I hope you enjoyed the course